VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT
CAPSTONE PROJECT
SEMESTER 251 ACADEMIC YEAR 2025-2026

SLURM AGENT SPECILIST

MAJOR: COMPUTER SCIENCE

SUPERVISOR(s): Assoc.Prof. Thoại Nam

——o0o——

STUDENT: Nguyễn Phúc Thanh Danh - 2252102

HO CHI MINH CITY, December 2024

Instructor's Signature

_____ Date: _____

Assoc.Prof. Thoại Nam (Instructor)
Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Assoc.Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2024

Authors,

Acknowledgement

We would like to express my appreciation to Assoc.Prof. Thoại Nam for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

High-Performance Computing (HPC) clusters have become essential infrastructure for scientific research, engineering simulations, and large-scale data processing. The Slurm Workload Manager is widely used across HPC environments and provides sophisticated resource management capabilities, but its command-line interface can create usability barriers for users who need to inspect queues, diagnose jobs, submit workloads, or perform controlled state-changing operations. This thesis addresses these challenges by designing and implementing an intelligent agent system that provides a natural language interface for Slurm cluster management.

The proposed Slurm Agent is a domain-specific control and assistance system for translating natural-language intent into concrete Slurm operations. Rather than treating an LLM as a general chatbot, the system combines typed Slurm tools, an Observer/Operator workflow with human confirmation for state-changing actions, local Slurm documentation retrieval, stateful confirmation handling, and trace-oriented evaluation. The implementation uses a React + Vite presentation layer, a session-aware FastAPI agent backend, an MCP protocol/tool layer, and real or mock Slurm infrastructure, but these frameworks serve as the substrate for Slurm-specific safety, grounding, and evaluation mechanisms.

This thesis makes six main contributions. **(1)** It defines a broad natural-language Slurm operation surface spanning monitoring, diagnosis, submission, job control, accounting, QoS/resource limits, reservations, scheduler health, configuration inspection, documentation support, and edge-case safety. **(2)** It implements a safety-centered Observer/Operator workflow that separates read-only analysis from state-changing execution through structured handoffs, human-in-the-loop confirmation, pending-action persistence, dynamic tool filtering, and session-scoped cleanup. **(3)** It develops a grounded Slurm assistance layer that combines live MCP command results for cluster state with local official-documentation retrieval for command syntax, state/reason-code interpretation, and policy-like explanations, reducing reliance on model memory alone. **(4)** It contributes a 3,135-case curated Slurm benchmark dataset with all eleven categories balanced at 285 cases each, including added coverage for fairshare, scheduler health, configuration inspection, GPU/GRES behavior, job arrays, dependencies, QOS and resource-limit diagnosis, reservations, reports, licenses, topology, federation, burst buffers, triggers, local documentation retrieval, and administrative safety workflows. **(5)** It provides an architecture-aware evaluation stack and dashboard that run live agent requests against resettable mock scenarios, collect traces for tool usage, routing, confirmation behavior, response quality, and source-to-target state transitions, and optionally apply an LLM-as-judge component. **(6)** It explores domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B) using QLoRA with role-scoped tool exposure, distilling the commercial model's evaluation traces into a local alternative that eliminates API dependency while preserving tool-calling and safety behavior.

The implementation demonstrates the feasibility of a stateful, tool-grounded natural language interface for HPC operations while making clear that production deployment requires continued safety hardening, broader paraphrase evaluation, and validation on real clusters with

site-specific policies. The fine-tuning investigation further demonstrates that domain-adapted open-weight models can serve as viable local alternatives to commercial APIs for structured tool-calling tasks, reducing operational cost and data-privacy concerns. The system provides a practical foundation for future research on AI-assisted cluster operations, especially where usability, traceability, and controlled execution must be balanced.

Keywords: High-Performance Computing, Slurm, Large Language Models, Model Context Protocol, Multi-Agent Systems, Human-in-the-Loop, Natural Language Interface, Parameter-Efficient Fine-Tuning

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CLI	Command-Line Interface
CoT	Chain-of-Thought
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSON-RPC	JavaScript Object Notation Remote Procedure Call
LLM	Large Language Model
MCP	Model Context Protocol
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RLHF	Reinforcement Learning from Human Feedback
SDK	Software Development Kit
SQL	Structured Query Language
Slurm	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
TLS	Transport Layer Security
UI	User Interface
URI	Uniform Resource Identifier

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Motivation

High-Performance Computing (HPC) has become an essential infrastructure in modern scientific research, engineering simulations, and large-scale data processing. HPC clusters enable researchers to tackle computationally intensive problems that would be infeasible on conventional computing systems. From climate modeling and genomics to machine learning and financial simulations, HPC systems provide the computational power necessary to drive innovation across multiple domains.

Slurm Workload Manager, originally introduced as the Simple Linux Utility for Resource Management, has emerged as a widely deployed workload manager for HPC clusters [48, 36]. SchedMD reports that Slurm is used at government laboratories, universities, and companies worldwide, and that it manages workloads for over half of the top 10 systems in the TOP500 list [34]. Slurm provides sophisticated mechanisms for job scheduling, resource allocation, and cluster monitoring, enabling efficient utilization of computational resources across thousands of nodes. The system's flexibility and scalability have made it the preferred choice for many academic institutions and commercial organizations operating large-scale computing facilities.

Despite its power and flexibility, HPC user-support literature highlights recurring needs for training, documentation, and assistance when researchers interact with complex computing environments [33]. In Slurm-based clusters, this challenge is reflected in a broad command-line interface (CLI) comprising numerous commands, each with extensive sets of options and parameters [36]. Users must master commands such as `squeue`, `sinfo`, `sbatch`, `scontrol`, and `sacct`, along with their associated flags and output formats. This complexity is compounded by the need to write job submission scripts in Bash, which requires additional technical knowledge beyond the domain expertise of many researchers.

The learning curve associated with Slurm administration and usage creates several operational challenges. System administrators spend considerable time training new users, while researchers face delays in their work as they navigate the intricacies of job submission and monitoring. Common tasks such as checking job status, understanding resource availability, or debugging failed jobs often require consulting extensive documentation or seeking expert assistance.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation [6, 24]. These models can interpret complex user queries, reason about multi-step tasks, and generate appropriate responses or actions. The emergence of LLM-based agents—systems that combine language models with tool-calling capabilities—has opened new possibilities for creating intelligent interfaces to complex

systems [46, 37].

The Model Context Protocol (MCP), developed by Anthropic, provides a standardized approach for connecting LLMs to external tools and data sources [3, 4]. MCP enables the creation of modular, reusable integrations that can expose system functionality to AI agents through explicit tool interfaces. This architectural pattern separates the concerns of tool implementation from agent logic, facilitating the development of sophisticated AI-assisted applications.

This research is motivated by the opportunity to bridge the gap between Slurm's powerful capabilities and user accessibility through an intelligent conversational interface. By leveraging LLM agents connected to Slurm through MCP, we can create a system that allows users to manage cluster resources using natural language queries, eliminating the need to memorize complex command syntax and reducing the cognitive load associated with HPC operations.

1.2 Scope and Goals

1.2.1 Project Goals

The primary objective of this research is to design, implement, and evaluate an intelligent agent system for Slurm cluster management. The project aims to make Slurm more accessible through natural-language interaction, but it also contributes the evaluation infrastructure needed to measure whether such an agent behaves correctly and safely. The following goals guide the development of this solution:

Goal 1: Natural Language Interface for Broad Slurm Operations. The system shall enable users to perform a broad set of Slurm operations through natural language queries. Users should be able to inspect jobs, partitions, nodes, accounting records, QoS and resource limits, reservations, scheduler health, submit workloads, and perform human-confirmed job-control operations without explicit knowledge of Slurm command syntax. The agent must interpret user intent accurately and translate queries into appropriate Slurm commands or documentation lookups.

Goal 2: Slurm-Specific Observer/Operator Control Flow. The system shall implement a specialized agent workflow where read-only observation is separated from state-changing job management. Documentation retrieval, data visualization, and web search fallback should remain read-only support capabilities, while handoffs to state-changing execution must carry explicit action intent, target scope, and safety context.

Goal 3: Risk-Aware Slurm Tool Integration via MCP. The system shall utilize the Model Context Protocol to provide standardized tool interfaces between the LLM agents and Slurm infrastructure. The MCP server should expose Slurm functionality as discrete, typed tools, while the agent layer classifies tools by operational risk and routes state-changing actions through confirmation before command execution.

Goal 4: Curated Slurm Benchmark Dataset. The project shall construct a broad, scorable Slurm benchmark dataset for evaluating natural-language cluster operations. The active dataset

shall cover monitoring, diagnosis, action, bulk operations, submission, safety, accounting, documentation, domain-specific Slurm workflows, multi-step reasoning, and edge cases. Each case should include expected tool behavior, routing expectations, confirmation policy, and source-to-target state information where applicable. The current active benchmark contains 3,135 curated cases, balanced across eleven categories with 285 cases each.

Goal 5: Domain-Adapted Fine-Tuning. The project shall explore whether a smaller open-weight model can be domain-adapted to replicate the tool-calling and safety behavior of the commercial baseline. The approach uses parameter-efficient fine-tuning (QLoRA) on traces distilled from the evaluation framework, with role-scoped tool exposure to prevent cross-role hallucination. The resulting model should operate entirely on local infrastructure, eliminating API cost and data-privacy concerns for production HPC environments.

1.2.2 Project Scope

The scope of this project encompasses the design, implementation, and evaluation of the Slurm Agent system. The following boundaries define what is included and excluded from the project:

In Scope:

- Development of an MCP server exposing Slurm functionality as tools
- Implementation of an Observer/Operator workflow with Slurm-specific risk separation and confirmation routing
- Design and implementation of a web-based frontend interface
- Integration of predefined chart artifact rendering for selected Slurm summaries
- Implementation of local Slurm documentation retrieval and web search fallback for documentation assistance
- Construction of a 3,135-case curated Slurm benchmark dataset with balanced categories
- Evaluation support through scripts, resettable mock scenarios, result artifacts, and dashboard monitoring
- Testing on a single-node Slurm cluster configuration and resettable mock Slurm scenarios
- Domain-adapted fine-tuning of an open-weight model using parameter-efficient methods (QLoRA) with comparative evaluation against the commercial baseline

This project focuses on demonstrating the feasibility of a tool-grounded Slurm agent with explicit confirmation for state-changing actions and on contributing the dataset and evaluation system needed to measure that behavior. The intended outcome is therefore both a working natural-language Slurm assistant and a reproducible benchmark foundation for future research in AI-assisted HPC operations.

1.3 Challenges and Solutions

Reliability and Accuracy

Challenge: Ensuring reliability and accuracy in a natural-language Slurm agent is difficult because Slurm commands depend on exact syntax, object identifiers, flags, partitions, accounts, and current scheduler state. A generated answer may sound plausible while still selecting the wrong tool, referring to the wrong job, misunderstanding a pending reason, or confusing documentation knowledge with live cluster facts.

Solutions: The system grounds user requests in typed MCP tools instead of free-form shell generation. The agent uses a ReAct-style execution pattern [47], but the important project-specific layer is the Slurm grounding around it: live state is retrieved through Slurm tools, command and state explanations are supported by local official-documentation retrieval, and under-specified requests can trigger clarification rather than forced execution.

Safety in Cluster Operations

Challenge: Slurm is a shared operational environment. Read-only inspection is low risk, but cancelling jobs, holding jobs, changing node state, modifying reservations, or updating accounting state can interrupt users and affect shared resources. These actions cannot be handled like ordinary chatbot responses.

Solutions: The system separates read-only observation from state-changing operation through an Observer/Operator workflow. The Observer handles inspection, diagnosis, accounting/resource-limit queries, reservation checks, documentation retrieval, and web-search fallback. The Operator handles state-changing work through structured handoffs and a pending-action workflow that requires explicit user confirmation before execution. Dynamic tool filtering keeps sub-agents within their intended responsibility boundaries, and MCP parameter validation prevents arbitrary shell-command construction.

Data Construction for Evaluation

Challenge: Evaluating an agentic Slurm assistant requires more than a small list of example prompts, but there is no ready-made public dataset that covers the exact Slurm workflows and tool behavior needed for this project. The benchmark data therefore has to be manually designed as synthetic scenarios. It must cover read-only monitoring, diagnosis, submission, job control, accounting, documentation questions, multi-step workflows, and edge cases, while remaining scorable through clear expectations for tools, routing, confirmation, response evidence, and state changes where applicable.

Solutions: The project manually constructs a synthetic 3,135-case scenario-grid benchmark across eleven balanced categories. Each case specifies expected tool behavior, routing behavior, confirmation policy, keywords, and source-to-target state behavior where applicable. The dataset is validated against supported tools and mock Slurm state so that benchmark failures can be interpreted as agent behavior rather than inconsistent ground truth.

Diversity of Slurm Workflows

Challenge: Slurm administration spans many domains, including jobs, nodes, partitions,

accounting, QoS, fair-share, reservations, configuration, scheduler health, job arrays, dependencies, licenses, reports, topology, and selected administrative actions. A narrow prototype using only `squeue` and `scancel` would not represent the operational breadth expected from an HPC assistant.

Solutions: The MCP tool layer exposes a broad Slurm operation surface, and the benchmark mirrors that breadth across balanced categories. Live or mock MCP command results remain the source of truth for cluster-state questions, while local Slurm documentation retrieval supports reference-heavy questions. This keeps live operational facts separate from documentation knowledge and reduces dependence on model memory alone.

State and Context Isolation

Challenge: Conversation state is useful for real users, but it can contaminate automated evaluation if previous test cases leak into later cases. Follow-up prompts need session memory, yet benchmark cases must start from clean context and known source state. Large artifacts or verbose traces can also pollute future model context if stored as ordinary conversation text.

Solutions: The backend maintains session-scoped conversation state and pending actions for interactive use, while evaluation runs use deterministic isolated sessions. Mock Slurm scenarios can be reset before each case, and parallel evaluation workers use separate session identifiers and worker-specific MCP endpoints. Chart artifacts and large generated payloads are separated from ordinary model context to reduce unnecessary token usage.

Traceable Agent Evaluation

Challenge: Text-only evaluation is insufficient for a tool-using agent. A response can explain the correct operation while still using the wrong tool path, skipping a required handoff, failing to request confirmation, or producing the wrong state transition in the simulator.

Solutions: The evaluator sends prompts through the live backend, records tool and routing traces, applies confirmation logic, and checks structural metrics such as tool recall, routing match, HITL behavior, keyword coverage, and state matching. This makes failures inspectable and repeatable, allowing the project to measure architecture-level behavior rather than relying only on final response fluency.

1.4 Project Structure

The report is divided into seven chapters, with a brief overview of each chapter as follows:

- **Chapter 1 - Introduction:** This chapter introduces the project topic, explains the motivation for a natural-language Slurm assistant, defines the scope and objectives, discusses the main challenges, and outlines the adopted solutions.
- **Chapter 2 - Theoretical Background:** This chapter presents the foundational knowledge required for the project, including Slurm workload management, Large Language Models, agent architectures, and the Model Context Protocol.

- **Chapter 3 - Related Tools and Works:** This chapter reviews existing HPC management interfaces, LLM agent frameworks, and MCP-based tool integrations, then positions the project relative to those works.
- **Chapter 4 - Proposed Approaches:** This chapter analyzes and designs the Slurm Agent system, including requirements, system architecture, component responsibilities, and the Observer/Operator workflow.
- **Chapter 5 - Implementation:** This chapter describes the implementation of the main system components: the MCP Slurm tool server, the agent backend, the frontend interface, and supporting evaluation infrastructure.
- **Chapter 6 - Experiments:** This chapter presents the testing approach, benchmark design, evaluation criteria, and experimental results of the Slurm Agent system.
- **Chapter 7 - Conclusion:** This chapter summarizes the project outcomes, discusses limitations, and outlines future development directions.

Chapter 2

Theoretical Background

2.1 Slurm Workload Manager

Slurm Workload Manager is an open-source cluster management and job scheduling system for Linux-based high-performance computing environments [48, 36]. It is widely deployed in research and production HPC clusters, including large systems represented in the TOP500 list [34]. This project uses Slurm as the target operational domain because it exposes a rich command surface for monitoring, diagnosis, submission, accounting, and controlled cluster administration.

2.1.1 Architecture Overview

Slurm follows a controller-and-worker architecture. The central controller maintains cluster state and scheduling decisions, while node-level daemons execute and monitor workloads.

The **slurmctld** daemon is the central controller. It tracks nodes, partitions, jobs, reservations, and resource allocations. It accepts job submissions, evaluates pending work, applies scheduling policy, and dispatches jobs to available compute resources. In production clusters, a backup controller can be configured for high availability.

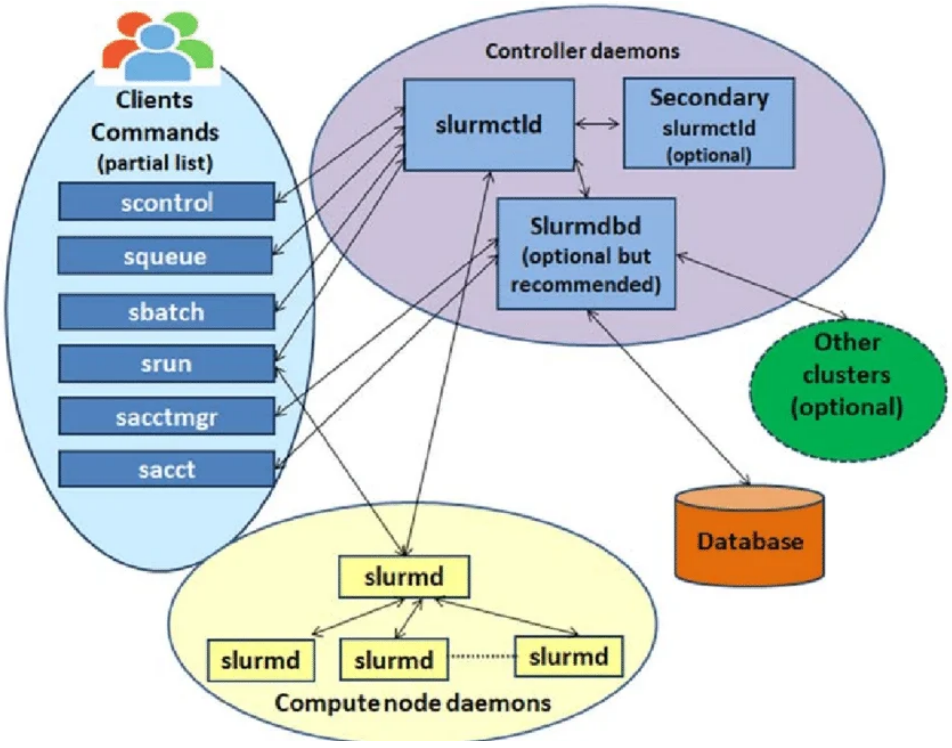

Figure 2.1: Typical Resource Management using Slurm [1]

The **slurmd** daemon runs on each compute node. It launches job steps, monitors local ex-

ecution, reports node state to the controller, and handles job control signals. This makes the compute node visible to the scheduler while preserving local execution responsibility.

The **slurmdbd** daemon provides the accounting service when enabled. It stores job history, user associations, account usage, fair-share information, and reporting data in a database. These records are important for administrative workflows, quota analysis, and historical job diagnosis.

2.1.2 Core Operational Objects

Slurm operations are built around several core objects:

Nodes are compute servers with resources such as CPUs, memory, GPUs, and generic resources. Node state matters operationally: an idle node can accept work, an allocated node is running jobs, a drained node is intentionally unavailable for new work, and a down node is unavailable because of failure or administrative action.

Partitions are logical groups of nodes. They behave like queues and encode access rules, limits, default policies, and intended usage patterns. Users often need to know which partition matches a job's resource requirements and policy constraints.

Jobs are resource allocation requests. A job includes requested resources, time limits, partition, account or QoS settings, script or command, and optional dependencies. Jobs move through states such as pending, running, completed, failed, cancelled, held, or suspended.

Steps are execution units inside an allocated job. A job may contain one or more steps, and step-level inspection is useful for understanding running work, CPU usage, memory behavior, and launched parallel tasks.

Accounts, QoS, and reservations represent policy and access constraints. They affect who can submit work, how resources are charged, which jobs receive priority, and when nodes are reserved for maintenance or special workloads.

These objects are accessed through a broad command surface. Read-only commands such as `sinfo`, `squeue`, `sacct`, and `scontrol show` inspect partitions, jobs, accounting records, nodes, reservations, and configuration state. Other commands, including `sbatch`, `srun`, `salloc`, `scancel`, selected `scontrol` operations, and `sacctmgr`, can submit work or modify jobs, nodes, reservations, accounts, and QoS policies. The same command family may contain both safe inspection operations and high-impact administrative operations; for example, `scontrol show job` is read-only, while `scontrol update` can change job or node state. This distinction is central to the Observer/Operator design used later in the thesis.

2.1.3 Why Slurm Is Difficult to Use Through Natural Language

Slurm's command-line interface is expressive, but that expressiveness creates usability and safety challenges. Users must know command names, flags, output formats, state meanings, pending reasons, account/QoS concepts, and site-specific policy conventions. Many operations are also state-dependent: cancelling a running job, releasing a held job, or updating a reservation only makes sense when the referenced object exists and is in an appropriate state.

These properties make Slurm a useful but demanding domain for a natural-language agent. The agent must map user intent to the correct command surface, obtain live cluster facts when needed, distinguish documentation questions from operational questions, and require explicit confirmation before state-changing actions are executed.

2.2 Large Language Models

Large Language Models (LLMs) are neural networks trained to model and generate natural language from large text corpora. Recent LLMs can follow instructions, summarize technical material, reason over multi-step requests, and call external tools, which makes them useful as natural-language interfaces for complex systems [6, 24]. In this thesis, the LLM is not treated as an authority on live cluster state. Instead, it is used as an intent interpretation and orchestration component that must be grounded through external tools, documentation, and structured responses.

2.2.1 Transformer-Based Language Modeling

Most modern LLMs are based on the Transformer architecture introduced by Vaswani et al. [41]. The key mechanism is self-attention, which allows each token representation to attend to other tokens in the sequence and model long-range dependencies.

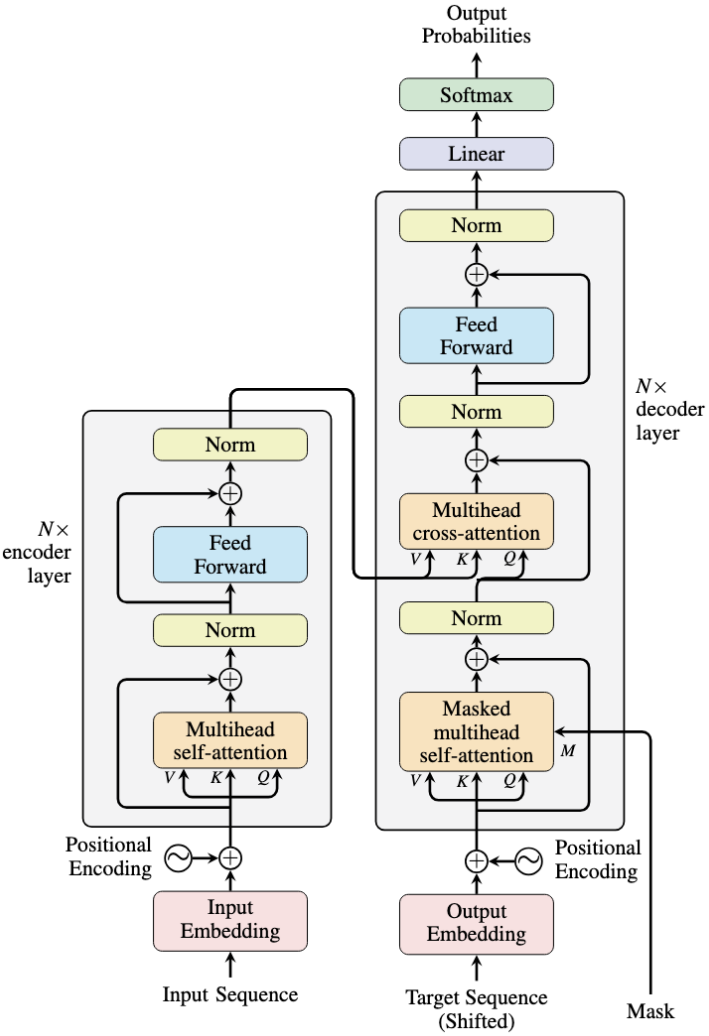

Figure 2.2: Transformer Architecture Components [41]

In self-attention, token representations are projected into query (Q), key (K), and value (V) vectors. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the key-vector dimension. Multi-head attention performs this computation across several learned projections, allowing different heads to capture different relationships in the text. Stacked attention and feed-forward layers then produce contextual representations used for next-token prediction.

Most LLMs are trained with an autoregressive objective, where the model predicts the next token from the preceding context:

$$P(x) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}) \tag{2.2}$$

This objective allows the model to learn linguistic patterns, factual associations, programming conventions, and domain terminology from training data. However, the generated output

remains probabilistic. A fluent answer can still be incomplete, stale, or unsupported by the current state of an external system.

2.2.2 Instruction Following and Alignment

Base language models are optimized for continuation, not necessarily for helpful task completion. Instruction tuning improves this behavior by training models on instruction-response examples. Reinforcement Learning from Human Feedback (RLHF) further adjusts model behavior according to human preferences for helpfulness, relevance, and safety [26]. Other alignment approaches, such as Constitutional AI, use explicit principles to critique and revise model outputs [5].

These techniques make LLMs better suited for conversational systems, but they do not remove the need for domain constraints. In a Slurm environment, an apparently correct response may still use the wrong partition, assume outdated job state, or omit an operational precondition. Therefore, alignment must be combined with tool grounding, parameter validation, and confirmation for actions that affect cluster state.

2.2.3 Tool Use and Structured Output

Tool use extends an LLM beyond static model knowledge. Instead of answering only from its parameters, the model can choose an external function, provide structured arguments, inspect the returned result, and continue the conversation using that result. This pattern is important for operational domains because live state must be queried from the system of record.

Function calling and structured output make tool use more reliable than free-form command generation. A developer defines available tools with names, descriptions, argument schemas, and required fields. The model then emits a structured call that software can validate before execution. For example, a natural-language request such as checking a user's queued jobs can be mapped to a job-query tool rather than a raw shell string.

Structured tool calls support three important properties in this thesis:

- **Grounding**: the agent can retrieve live Slurm state instead of relying on memorized or guessed information.
- **Control**: the application can restrict the available tool set according to the current interaction mode.
- **Validation**: typed parameters can be checked before a command is executed against the cluster interface.

2.2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) addresses a core limitation of LLMs: their parametric knowledge is frozen at training time and cannot reflect new documentation, version-specific

syntax, or domain-specific reference material [19]. RAG augments the generative model with a retrieval step that fetches relevant passages from an external corpus before generation, grounding the output in authoritative source material.

A typical RAG pipeline consists of three stages:

1. **Indexing**: documents are split into chunks and encoded into dense vector representations using a pre-trained embedding model such as Sentence-BERT [32].
2. **Retrieval**: at query time, the user query is embedded with the same model and the top-k most similar chunks are retrieved via cosine similarity.
3. **Generation**: the retrieved chunks are prepended to the LLM prompt as context, and the model generates an answer grounded in those passages.

Lexical vs. Semantic Retrieval

Lexical retrieval (e.g., BM25, TF-IDF) matches based on token overlap between query and document. It is fast and effective for keyword-rich queries but fails when the user paraphrases concepts without using the exact terminology present in documents.

Semantic retrieval encodes both queries and documents into a shared embedding space where cosine similarity captures meaning rather than surface form. For example, the query "how to chain jobs so one runs after another" can match documentation about `--dependency=afterok` even though no keywords overlap.

Hybrid Retrieval and Reciprocal Rank Fusion

Neither method dominates across all queries. Hybrid retrieval combines both by fusing their ranked result lists. Reciprocal Rank Fusion (RRF) [8] merges rankings without requiring score normalization:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{2.3}$$

where R is the set of retrieval systems and k is a smoothing constant (typically 60). Documents that rank highly in both lexical and semantic systems receive the highest fused score, while documents that appear in only one system are still surfaced.

This approach is well-suited for operational documentation retrieval where queries range from exact command names ("sbatch –parsable") to conceptual questions ("why is my job stuck").

2.2.5 Limitations in Operational Settings

LLMs introduce risks that are especially relevant to HPC operations. They may hallucinate non-existent commands, merge similar options from different tools, provide outdated documentation,

or infer an action that the user did not explicitly approve. They may also produce plausible explanations for failures without actually checking logs, accounting records, or scheduler state.

For these reasons, the agent architecture in this project separates language understanding from operational execution. The LLM interprets intent and decides which typed tool may be relevant, while external tools provide authoritative data and the application enforces confirmation before state-changing operations. This design uses the LLM for its strengths in language and orchestration while limiting its authority over the cluster.

2.2.6 Parameter-Efficient Fine-Tuning

Full fine-tuning of large language models updates all parameters, requiring GPU memory proportional to the full model size. For models with billions of parameters, this is impractical on commodity hardware. Parameter-Efficient Fine-Tuning (PEFT) methods address this by modifying only a small subset of parameters while keeping the base model frozen.

Low-Rank Adaptation (LoRA) [16] decomposes weight updates into low-rank matrices. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA adds a trainable decomposition:

$$W = W_0 + \Delta W = W_0 + BA \tag{2.4}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. Only A and B are trained, reducing trainable parameters from $d \times k$ to $r \times (d + k)$. A scaling factor α/r controls the magnitude of the adaptation. At inference time, the low-rank matrices can be merged back into the base weights with zero additional latency.

Quantized LoRA (QLoRA) [9] extends LoRA by quantizing the frozen base model to 4-bit precision (NormalFloat4) during training. The LoRA adapters remain in higher precision (bfloat16) for gradient computation. This reduces the memory footprint of the base model by approximately 4×, enabling fine-tuning of 14B-parameter models on a single 48 GB GPU. QLoRA introduces double quantization—quantizing the quantization constants themselves—for additional memory savings with negligible quality loss.

Knowledge Distillation transfers capabilities from a larger teacher model to a smaller student model. In the context of tool-calling agents, distillation can use the teacher's successful execution traces as training data: the student learns to reproduce the teacher's tool selections, argument formatting, and response patterns without requiring manually annotated data [14]. This is particularly effective when the teacher model's behavior is already validated through an evaluation framework.

2.3 Model Context Protocol

The Model Context Protocol (MCP) is an open protocol for connecting AI applications to external tools, resources, and reusable prompts [3, 4]. It provides a standard boundary between a language model application and the systems it needs to inspect or operate. In this thesis, MCP is

important because it allows Slurm operations to be exposed as typed tools instead of free-form shell commands.

2.3.1 Protocol Overview

MCP addresses a common integration problem in AI applications. Without a shared protocol, each application must implement its own custom connection to every database, API, command-line tool, or document source. This makes integrations difficult to reuse and makes behavior harder to inspect.

MCP uses a client-server model. A server declares the capabilities it provides, and a client connects those capabilities to an AI host application. Communication is based on JSON-RPC 2.0, which gives the protocol a structured request-response format that can be implemented across different programming languages.

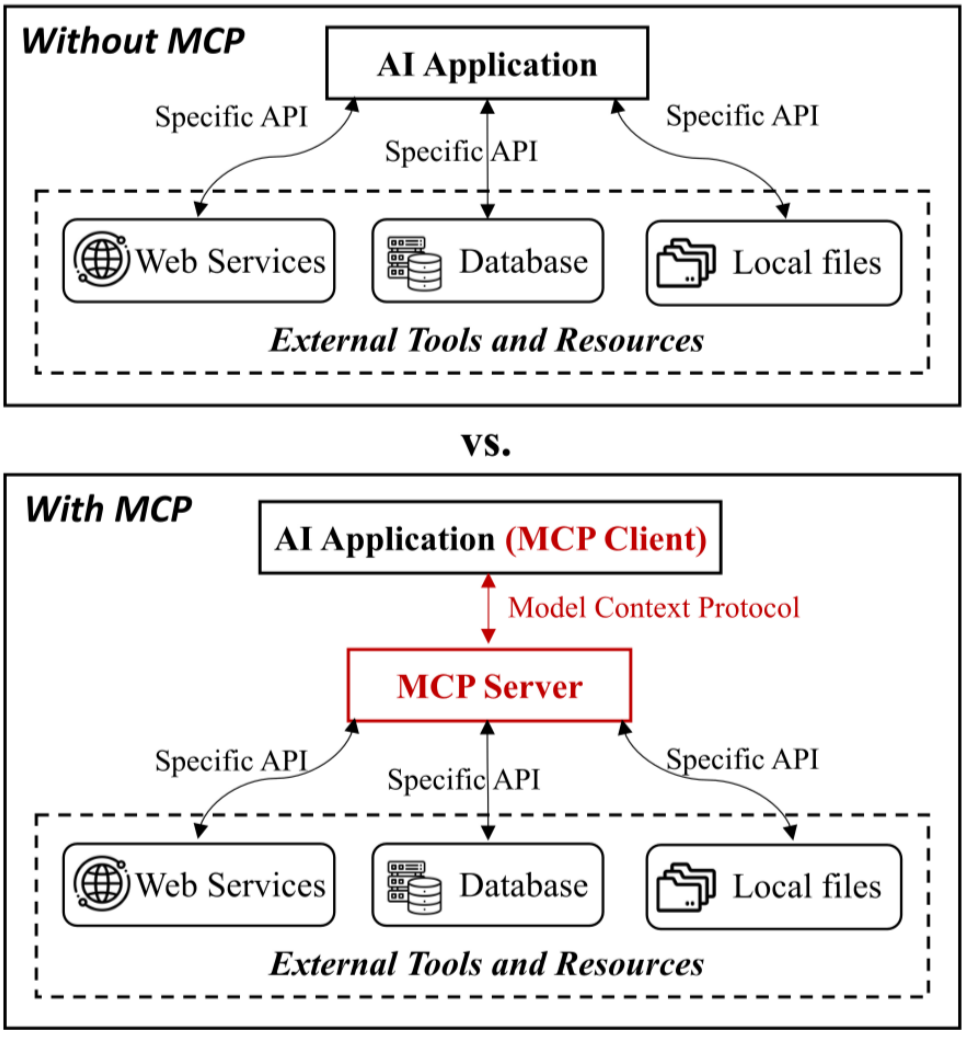

Figure 2.3: Model Context Protocol Architecture [15]

The protocol does not define the internal logic of a tool. Instead, it defines how the tool is described, discovered, invoked, and returned to the host application. This makes MCP suitable for wrapping existing systems such as Slurm command-line utilities.

2.3.2 Core Components

MCP architecture contains three main roles:

- **MCP Hosts**: applications that users interact with directly, such as chat interfaces, IDE extensions, or desktop assistants. The host manages the user interface and coordinates the model's access to external capabilities.
- **MCP Clients**: protocol clients that maintain connections between the host and one or more servers. They perform capability discovery, forward requests, and receive server responses.
- **MCP Servers**: domain-specific services that expose capabilities. In this project, a Slurm MCP server exposes selected scheduler operations as named tools with defined input schemas.

This separation improves reuse. The same Slurm-oriented server can be connected to different host applications, and the host does not need to know the implementation details of each Slurm command.

2.3.3 Capabilities

MCP servers can expose three categories of capability:

- **Resources**: readable data sources such as files, documents, logs, or generated status information.
- **Tools**: executable operations with names, descriptions, and input schemas.
- **Prompts**: reusable prompt templates for domain-specific workflows.

For Slurm Agent, tools are the main capability. They allow operations such as listing jobs, inspecting nodes, reading accounting data, or preparing a state-changing request to be represented as explicit function calls. Resources and prompts can support the interaction by providing documentation or task-specific guidance, but operational access is primarily controlled through tools.

2.3.4 Tool Definition and Invocation

MCP tools are described with metadata and JSON Schema input definitions. A tool declaration normally includes a tool name, a natural-language description, and a schema that specifies accepted parameters. When the model selects a tool, the client sends a `tools/call` request with the tool name and arguments. The server validates the input, performs the underlying operation, and returns a structured result or error.

This pattern is useful for Slurm because it avoids asking the model to construct arbitrary shell commands. Instead of generating a raw `squeue` or `scontrol` string, the model can call a specific tool such as a job-listing operation with typed filters. The server is then responsible for translating validated parameters into the appropriate Slurm command and formatting the result.

Typed tool invocation supports the safety model used in this thesis. Read-only tools can be made available for observation, while tools that may affect cluster state can be separated into a different interaction mode and require explicit user confirmation before execution.

2.3.5 Transport and Deployment

MCP can run over local or network transports. Local servers are commonly launched as subprocesses and communicate through standard input and output. Network deployments use HTTP-based transport so that a server can be hosted separately from the user-facing application.

The transport mechanism affects deployment, scaling, and process management, but it does not change the logical tool interface. A Slurm tool has the same schema and meaning whether the MCP server is launched locally during development or exposed through a network service in an integrated application.

2.3.6 Security Considerations

MCP provides structure, but it does not automatically make tool use safe. Security must be enforced by the server and the host application. Important concerns include:

- **Access control**: only expose operations that the current user or deployment is allowed to perform.
- **Input validation**: reject missing, malformed, or out-of-policy arguments before command execution.
- **Output control**: avoid exposing sensitive user, job, or accounting information unnecessarily.
- **Auditability**: log tool invocations and results for debugging and accountability.
- **User confirmation**: require explicit approval before actions that modify cluster state.

These concerns are especially important for HPC systems because a single operation can affect shared resources, queued jobs, and other users. Therefore, this project uses MCP as the structured integration layer, while confirmation flow and mode separation provide additional control over state-changing Slurm operations.

2.4 Agent Architectures and Frameworks

Building intelligent agents requires combining Large Language Models with structured reasoning patterns and tool integration frameworks. This section examines the ReAct agent architecture and the OpenAI Agents SDK, which form the foundation of the Slurm Agent implementation.

2.4.1 ReAct: Reasoning and Acting

The ReAct paradigm, introduced by Yao et al. [47], represents a significant advancement in LLM-based agent design. ReAct synergizes **reasoning** (generating verbal reasoning traces) with **acting** (taking actions that affect the environment), enabling agents to solve complex tasks that require both planning and external interaction.

The Think-Act-Observe Loop

ReAct agents operate through an iterative loop with three distinct phases:

1. **Think**: The agent generates a reasoning trace explaining its current understanding and intended next step. This internal monologue helps decompose complex problems and maintain task focus.

2. **Act**: Based on the reasoning, the agent selects and executes an action—typically invoking an external tool with specific parameters.

3. **Observe**: The agent receives the result of its action from the environment and incorporates this observation into its context for the next iteration.

This cycle repeats until the agent determines it has sufficient information to provide a final response, or encounters a termination condition.

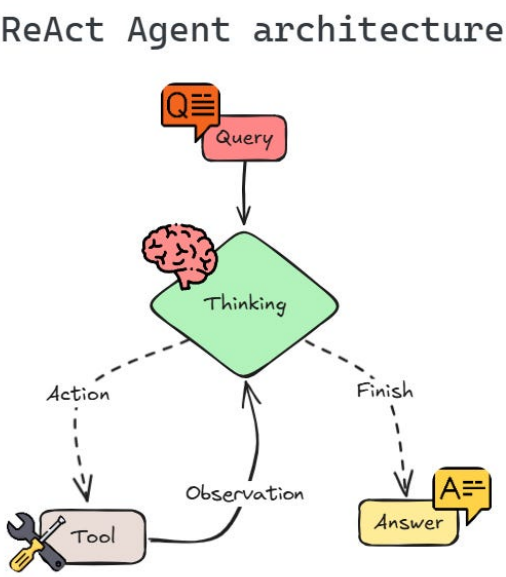

Figure 2.4: The ReAct Think-Act-Observe Loop

ReAct is suitable for HPC operations because: (1) Slurm diagnosis requires chaining multiple queries (queue state, priority, partition availability) before concluding; (2) the reasoning trace makes tool selections auditable; (3) observations can revise the agent's plan when initial assumptions are wrong.

2.4.2 OpenAI Agents SDK

The OpenAI Agents SDK [25] provides the execution framework used in this project. Its key abstractions are:

- **Agent**: An LLM with instructions, tools, and behavioral config. Separate agents isolate read-only observation from state-changing operations.
- **Runner**: Manages the execution loop—sending messages, processing tool calls, collecting results.
- **Tools**: Functions agents invoke to interact with external systems, with automatic parameter validation.
- **Handoffs**: Transfer control between specialized agents, passing conversation context.

The Runner's internal loop naturally implements ReAct: (1) collect messages, (2) send to LLM, (3) if tool calls, execute and loop; (4) if final text, return; (5) if handoff, transfer to target agent. The SDK also integrates with MCP servers, treating MCP tools as native agent tools.

Chapter 3

Related Tools and Works

3.1 Related Works

This chapter reviews work related to natural-language assistance for Slurm-based HPC operation.

3.1.1 HPC Management Interfaces

Slurm provides expressive CLI tools (`squeue`, `sacct`, `sinfo`, `sbatch`, `scancel`) but assumes scheduler expertise [48, 36]. Portals such as Open OnDemand [17] and JupyterHub [18] reduce barriers but still require manual parameter selection. Monitoring tools (Open XDMoD [27], Prometheus/Grafana [29, 13]) are observational and do not convert requests into validated actions.

3.1.2 LLM-Based HPC Systems

Recent work integrates LLMs with HPC, though with different goals. Chat AI [10] deploys LLMs as services on Slurm-backed GPUs but does not use LLMs to *manage* the cluster. LangChain-Parsl [2] connects an LLM agent to HPC workflows via the Parsl framework but lacks HITL safety and native Slurm tool integration. Fujitsu's AI Computing Broker [12] optimizes GPU allocation across clusters at the resource-brokering layer.

Table 3.1: Comparison of LLM-Based HPC Systems

System	NL Mgmt	Tool Calls	HITL	Slurm-Native
Chat AI	No	No	N/A	Yes
LangChain-Parsl	Yes	Yes	No	No (Parsl)
Fujitsu ACB	No	No	N/A	Partial
Slurm Agent	Yes	Yes	Yes	Yes

3.1.3 LLM Agents and Tool-Augmented Reasoning

ReAct combines reasoning traces with actions [47]. Toolformer demonstrates tool-calling patterns [37]. Agent surveys identify planning, memory, tool use, and multi-agent coordination as key design dimensions [7, 46]. Frameworks such as the OpenAI Agents SDK and AutoGen provide implementation support including tool invocation, streaming, and handoff patterns [25, 45]. These are general-purpose; a Slurm assistant must additionally know which tools are read-only, which require confirmation, and how to interpret scheduler state.

3.1.4 Protocol-Based Tool Integration

The Model Context Protocol (MCP) defines a standard client-server interface for exposing tools to AI applications [3, 4]. This project applies MCP to a Slurm-specific operational setting with read/write tool separation, mock scenarios for evaluation, and traceable behavior.

3.1.5 Natural Language to Structured Operations

Semantic parsing (Seq2SQL [52], Spider [49], RAT-SQL [42]) maps language to structured queries. For Slurm, the operation space is a set of scheduler tools with parameters, states, and safety policies rather than a database schema. This makes the system closer to grounded operation generation with action risk management.

3.1.6 Safety and Evaluation

Alignment methods [5, 26] improve assistant behavior but cannot guarantee correctness in live systems. LLM-as-judge evaluation [51] does not verify tool-grounded workflows. The benchmark in this thesis therefore checks tool selection, routing, HITL compliance, and state transitions across 3,135 Slurm-specific scenarios.

Chapter 4

Proposed Approaches

4.1 Proposed Slurm Agent Overview

The proposed system is a stateful Slurm Agent for translating natural-language operational requests into grounded scheduler interactions. Its purpose is to study how an LLM-based agent can interpret Slurm intent, choose typed tools, separate read-only inspection from state-changing operations, and produce traces that can be evaluated against expected behavior. The user-facing interface is only an entry point; the main design object is the agent control flow and the evaluation method around it.

4.1.1 System Objective

The system sits between a natural-language request and the Slurm command surface. A request such as "why is my job pending?", "show failed jobs for this account", or "cancel job 12345" is not executed as a free-form shell command. Instead, the agent maps the request onto a constrained set of MCP tools, retrieves current or simulated cluster state, and decides whether the task is observational, diagnostic, documentation-oriented, or state-changing.

This objective creates three design requirements. First, the agent must ground claims in Slurm evidence rather than model memory alone. Second, commands that modify cluster state must pass through an explicit confirmation path. Third, the runtime must expose enough trace information to evaluate tool choice, routing, confirmation behavior, and state-transition consistency.

4.1.2 Core Agent Capabilities

The proposed system combines the following capabilities:

- **Slurm intent interpretation**: map natural-language requests to monitoring, diagnosis, documentation, submission, accounting, job-control, or safety-sensitive workflows.
- **Observer/Operator separation**: route read-only inspection to the Observer path and state-changing operations to the Operator path with confirmation control.
- **MCP tool grounding**: expose Slurm commands as typed tools with explicit parameters instead of asking the model to generate arbitrary shell commands.
- **Documentation and web-search grounding**: retrieve local official Slurm documentation for command syntax, state meanings, reason codes, and policy-like explanations, with

web search as a fallback for external troubleshooting material not covered by the local corpus.

- **Traceable execution**: record tool calls, handoff behavior, pending actions, response evidence, and state effects so each case can be inspected and scored.
- **Dataset-driven evaluation**: run manually constructed benchmark cases against resettable mock Slurm states and compare observed behavior with expected tools, routing, HITL labels, and target-state semantics.

4.1.3 Operational Flow

A typical request follows this control path:

1. A user or evaluator submits a natural-language Slurm request through the runtime API.
2. The backend loads the session context, model configuration, MCP connection, and any pending-action state.
3. The main ReAct agent interprets the request and decides whether to answer from context, retrieve local documentation, use web search for external troubleshooting context, inspect cluster state, or hand off to an action path.
4. The Observer path performs read-only tool calls for queue, node, accounting, reservation, configuration, documentation, web-search, or diagnostic evidence.
5. The Operator path prepares state-changing actions such as submission, cancellation, hold, release, requeue, or selected updates, but execution is delayed until confirmation is received.
6. Tool outputs, handoff decisions, pending-action events, final response text, and state effects are emitted as structured trace evidence.
7. During evaluation, the trace is compared with benchmark ground truth for tool recall, routing match, HITL match, response evidence, and source-to-target state behavior.

This flow keeps the thesis focus on agent behavior rather than interface design. The same runtime path can support manual interaction and automated evaluation, which makes implementation choices directly testable in the scenario-grid benchmark.

4.2 Requirements Overview

This section defines the functional and non-functional requirements that guide system design. Requirements are derived from the project goals stated in Section 4.1 and the operational needs of HPC administrators and researchers.

4.2.1 Functional Requirements

Table 4.1: Functional Requirements

ID	Requirement
FR1	The system shall interpret natural-language Slurm queries, including variations in phrasing and implicit context, and map them to appropriate typed tool calls.
FR2	The system shall retrieve live cluster state (queues, nodes, partitions, accounting, reservations, scheduler diagnostics) via read-only Slurm tools and present results grounded in tool output.
FR3	The system shall execute state-changing operations (submit, cancel, hold, release, requeue, node control, account management) only after explicit user confirmation.
FR4	The system shall retrieve relevant Slurm documentation from a local corpus using hybrid retrieval (lexical + semantic), with web search as a fallback when the local corpus does not cover the topic.
FR5	The system shall maintain multi-turn conversation context, enabling follow-up questions and contextual references within a session.
FR6	The system shall provide real-time feedback during execution through structured status events, making tool selections and intermediate results visible to the user.
FR7	The system shall support deterministic evaluation against mock cluster states, producing behavioral traces that can be compared to expected tool selections, routing decisions, and state transitions.

4.2.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

ID	Requirement
NFR1	**Safety**: The system shall never execute cluster-mutating commands without explicit confirmation. The confirmation mechanism shall be enforced at the tool layer, independent of LLM behavior.
NFR2	**Accuracy**: Responses shall be grounded in tool output. The system shall not fabricate cluster state, job IDs, node names, or resource figures.
NFR3	**Latency**: Simple read-only queries shall complete within interactive response times. Long-running workflows shall emit incremental status events.
NFR4	**Extensibility**: New Slurm tools shall be addable via the MCP server without modifying the agent core. Tool schemas are discovered at runtime.
NFR5	**Context Efficiency**: The system shall avoid unnecessary growth of LLM context by lazy-loading documentation, capping retrieval output size, and separating large artifacts from conversational memory.
NFR6	**Reproducibility**: Evaluation runs shall be deterministic given the same mock state, model, and test case. Provider and model shall be configurable per session.

4.2.3 Requirements Traceability

Table 4.3 maps each requirement to the project goals defined in Section 4.1.

Table 4.3: Requirements Traceability to Project Goals

Requirement	**G1** *NL Slurm Access*	**G2** *Control Flow*	**G3** *MCP Tools*	**G4** *Dataset Eval*
FR1: Intent Interpretation	✓			
FR2: Cluster State Retrieval	✓		✓	
FR3: Confirmed State Changes	✓	✓	✓	
FR4: Documentation Retrieval	✓		✓	
FR5: Multi-Turn Conversation	✓	✓		
FR6: Real-Time Feedback		✓		✓
FR7: Deterministic Evaluation			✓	✓
NFR1: Safety		✓		✓
NFR2: Accuracy	✓			✓
NFR3: Latency	✓			
NFR4: Extensibility			✓	
NFR5: Context Efficiency		✓		
NFR6: Reproducibility				✓

4.2.4 Constraints

Technology: Python backend, MCP-based tool layer, local Ollama models for reproducibility with optional remote providers for development and comparison.

Environment: Evaluation uses mock Slurm states exposed by the MCP server. Real deployment assumes standard Slurm command availability and site permissions.

Scope: The system targets broad Slurm assistance (monitoring, diagnosis, submission, job control, accounting, documentation). High-risk operations are gated by explicit confirmation rather than unrestricted automation.

4.3 System Architecture

This section presents the overall architecture of the Slurm Agent system, describing the major components, their interactions, and the rationale behind architectural decisions. The design uses common web and agent frameworks, but the architectural contribution is the Slurm-specific control plane built on top of them: broad Slurm tool coverage, read/write separation with confirmation, local documentation grounding, and state-resettable evaluation.

4.3.1 Architecture Overview

The Slurm Agent is organized as a four-layer stack. At the top, a thin **Interface Layer** accepts natural-language requests and relays confirmation decisions without embedding any domain

logic. Below it, the **Agent Layer** houses the Observer/Operator multi-agent backend that performs ReAct reasoning, retrieves documentation, and enforces safety routing—the Observer handles read-only inspection and decides when to hand off to the Operator for state-changing operations. The **Protocol Layer** exposes each Slurm operation as a typed MCP tool with parameter validation, isolating the agent from raw command-line construction. At the bottom, the **Infrastructure Layer** provides the actual execution targets—either a live Slurm cluster or a mock scenario engine—alongside the local documentation corpus and external services.

This separation ensures that the boundary between reading cluster state and mutating it is an explicit architectural seam rather than an implicit convention. Read-only inspection flows entirely within the Observer path, while any state-changing operation must cross into the Operator path and pass through a confirmation gate before reaching the MCP tool layer.

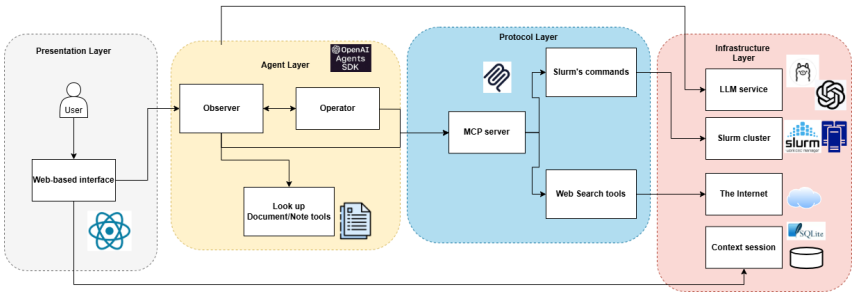

Figure 4.1: Slurm Agent System Architecture

4.3.2 Design Rationale

The main design decision is to treat Slurm interaction as a controlled operation surface rather than as open-ended command generation. A general chatbot can explain commands, but a cluster assistant must also decide when to inspect live state, when to consult documentation, when to ask for clarification, and when to refuse or delay execution until a user confirms intent. The architecture therefore combines three forms of grounding:

- **Operational grounding**: current cluster facts come from Slurm tools exposed through MCP.
- **Documentation grounding**: command syntax, reason-code meanings, and policy-like explanations come from local Slurm documentation retrieval.
- **Evaluation grounding**: behavior is tested against resettable source states and expected target states instead of relying only on subjective response quality.

The architecture is also designed to support incomplete or ambiguous user requests. When the user says a phrase such as “cancel that job”, the system must determine whether a specific job is already known from context. If the target is unclear, the safe behavior is to ask for clarification. If the target is clear but the action changes cluster state, the safe behavior is to create a pending action and require confirmation. This is why session state and pending-action persistence are architectural components rather than UI conveniences.

4.3.3 Component Descriptions

Interface Layer

The Interface Layer is intentionally thin. It provides an entry point for manual prompts, automated evaluation prompts, and explicit confirmation decisions. This layer is responsible for:

- forwarding natural-language requests to the agent backend,
- presenting final responses and selected status events,
- relaying confirm/cancel decisions for pending actions,
- exposing traces needed for debugging and evaluation.

The interface is therefore support infrastructure, not the central contribution. Agent behavior is defined by the backend control flow and can be exercised either manually or through the automated evaluator.

Agent Layer

The Agent Layer implements the core reasoning and orchestration logic using a multi-agent architecture. The system employs a hierarchical structure with specialized agents:

- **Main ReAct Agent**: The primary orchestrator that implements the Think $\rightarrow$ Act $\rightarrow$ Observe reasoning loop. This agent maintains conversation context, interprets user intent, and delegates to specialized sub-agents based on the nature and risk of the request.
- **Observer**: Handles read-only information gathering operations including cluster status queries, job queue inspection, resource monitoring, accounting and resource-limit inspection, reservation checks, scheduler diagnostics, and documentation retrieval. This agent can chain multiple tool calls to gather comprehensive information.
- **Operator**: Manages operations that modify cluster state, such as job submission, cancellation, hold, release, requeue, and selected updates. This agent receives structured handoffs and enforces safety controls and confirmation requirements for dangerous operations.

The Observer can inspect queue state, node state, accounting records, reservations, fairshare, priority, scheduler diagnostics, and documentation. The Operator handles submission, cancellation, hold, release, requeue, update, reservation changes, node changes, and administrative actions. This division is central because incorrect execution can waste compute time, interrupt other users, or change shared cluster policy.

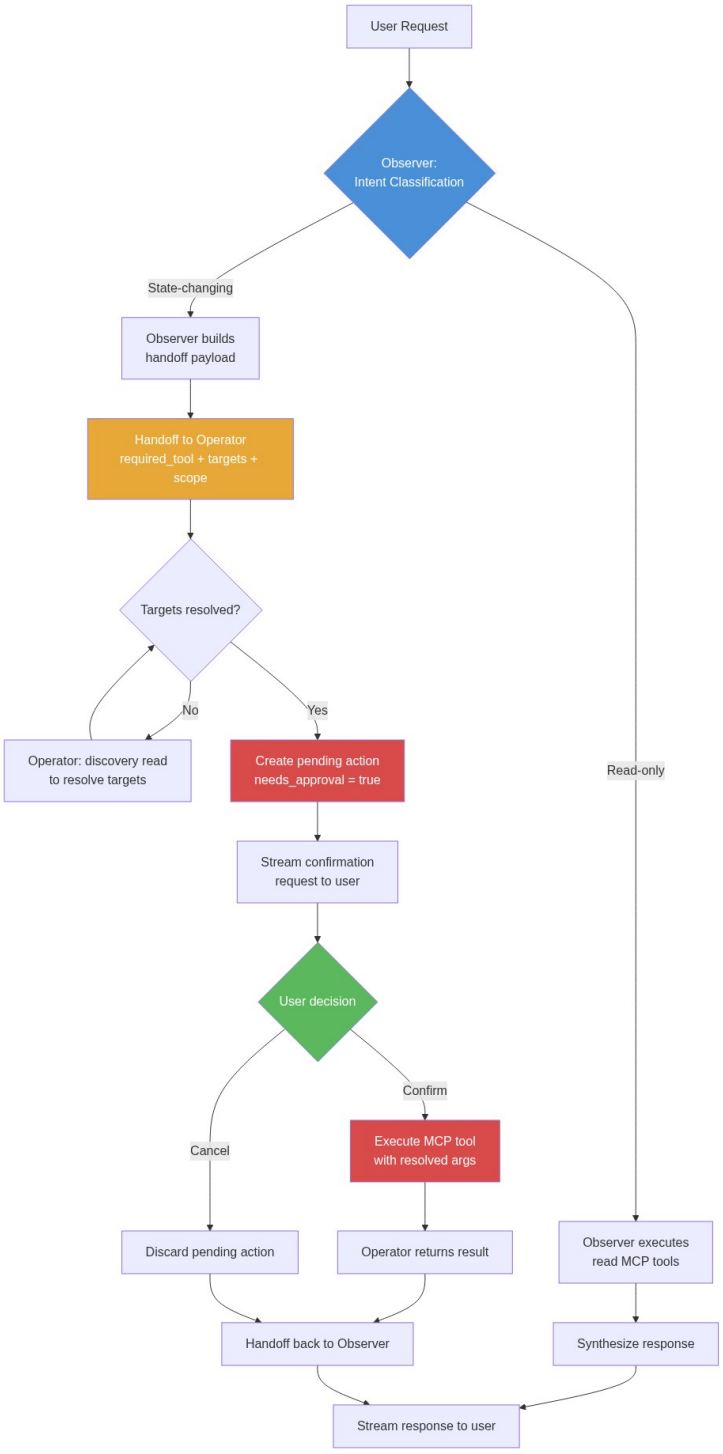

Figure 4.2: Observer/Operator Routing and Confirmation Flow

Protocol Layer

The MCP Protocol Layer wraps each Slurm command as a typed tool with parameter validation, operational semantics preservation, safe subprocess execution, and structured output formatting. It decouples agent logic from Slurm implementation details and provides a security boundary for command execution.

Evaluation Layer

The evaluation layer verifies agent behavior by controlling mock-state reset, sending prompts to the live backend, recording stream events, and comparing observed tools/handoffs/HITL against ground truth.

Infrastructure Layer

External systems: LLM provider (local Ollama or cloud API), Slurm cluster (real or mock), local documentation corpus, web search service, and SQLite context database.

4.3.4 Communication Patterns

Key patterns: (1) request-response for queries, (2) structured event stream for status/tool/trace output, (3) tool invocation via MCP, (4) confirmation interruption for dangerous operations, (5) trace reporting for evaluation scoring.

4.3.5 Data Flow

A typical query flows: (1) user submits request, (2) Agent Layer begins ReAct reasoning, (3) delegates to Observer or Operator, (4) sub-agent invokes MCP tools, (5) results propagate back, (6) agent synthesizes response with trace events. For dangerous operations, an additional confirmation step interrupts the flow before execution.

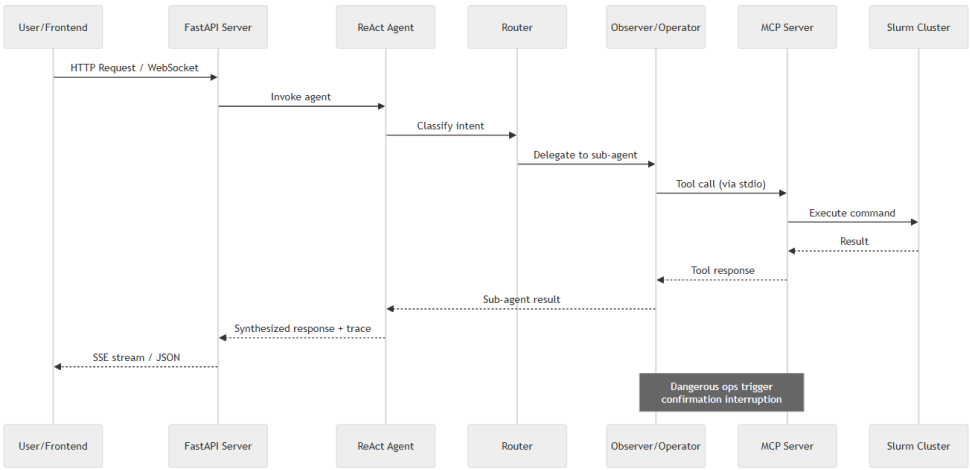

Figure 4.3: Request Lifecycle Data Flow

4.3.6 Deployment Architecture

The system supports local development (all components on one machine) and distributed deployment (interface, backend, and MCP server on separate hosts, with MCP on a Slurm management node).

4.4 Module Specifications

This section specifies the operational workflow of each functional module. Each module is described by its role, processing stages, and interaction contracts.

4.4.1 Observer Module

The Observer is the primary entry point for all requests and handles read-only operations. It follows a ReAct loop: (1) classify user intent, (2) select and invoke read-only MCP tools, (3) chain additional tool calls if evidence is insufficient, (4) synthesize a grounded response. If a state-changing operation is detected, the Observer produces a structured handoff containing the classified intent, gathered evidence, and target identifiers, then transfers control to the Operator.

The Observer accesses 40+ read-only tools spanning queue inspection (`squeue`), node status (`sinfo`), job history (`sacct`), scheduler diagnostics (`sdiag`, `sprio`), resource statistics (`sstat`, `sreport`), configuration display (`scontrol_show`), documentation (`lookup_slurm_docs`), and web search.

4.4.2 Operator Module

The Operator manages all state-changing operations. It is activated exclusively through Observer handoffs and enforces a Human-in-the-Loop (HITL) confirmation protocol before any destructive action.

Workflow: (1) Receive handoff with classified intent and targets. (2) Optionally verify current state with a discovery read. (3) Present the planned action to the user via a pending action event. (4) Block until explicit user confirmation or cancellation. (5) Execute the tool call upon confirmation. (6) Perform post-execution verification and return results.

Tool Classification: Tools are dynamically classified at startup into three categories—*analysis* (read-only, no confirmation needed), *safe* (information retrieval), and *dangerous* (state-modifying, always requires HITL). Any tool whose name or description implies state modification is classified as dangerous.

4.4.3 MCP Tool Layer

The MCP server provides 67 typed tools organized into functional groups. Table 4.4 shows the distribution across operational categories. The grouping reflects Slurm's own command

families: read-only monitoring tools dominate (suitable for the Observer), while job control, node management, and accounting tools require Operator-level confirmation before execution.

Table 4.4: MCP Tool Groups

Group	**Count**	**Examples**
Queue/Monitoring	10	`squeue`, `sinfo`, `sacct`, `sdiag`
Job Control	8	`sbatch`, `scancel`, `scontrol_hold`
Node Management	6	`scontrol_node`, power up/down
Reservations	4	create/delete/update reservation
Accounting	8	`sacctmgr` show/add/modify/delete
Cluster Admin	5	reconfigure, shutdown
Documentation & Web	3	`lookup_slurm_docs`, `web_search`
Other (triggers, charts, mock)	23	triggers, chart generators, test utilities
Total	**67**	

Dual Execution Mode: The server supports *real mode* (translates tool calls to Slurm CLI commands) and *mock mode* (operates on mutable in-memory state with five predefined scenarios). Mock mode enables deterministic evaluation from known starting states.

Parameter Validation: Before execution, the MCP layer validates job IDs, node names, partition names, and scans batch scripts for dangerous patterns, providing a safety layer independent of LLM reasoning quality.

4.4.4 Documentation Retrieval

A local corpus of 130 pages of official Slurm documentation enables grounded answers without relying on LLM parametric memory. The `lookup_slurm_docs` tool performs keyword-based lookup against an indexed corpus. When local docs are insufficient, the agent falls back to `web_search` and `fetch_web_content` for external sources.

4.4.5 Evaluation Pipeline

Each benchmark test case specifies: a natural-language request, source cluster state, target state, expected tools, routing decision, and HITL requirement. Scoring uses five weighted dimensions:

$$\text{Overall} = 0.30 \cdot \text{ToolRecall} + 0.25 \cdot \text{Routing} + 0.25 \cdot \text{HITL} + 0.10 \cdot \text{Keywords} + 0.10 \cdot \text{State} \quad (4.1)$$

A case passes when overall ≥ 0.80. Aggregate metrics include pass rate, behavioral agreement rate, and safety violation rate. An optional LLM-as-judge provides supplementary 1–5 quality scores without affecting pass/fail determination.

4.5 Large Language Model Architecture

4.5.1 ReAct Reasoning Loop

The agent follows the ReAct paradigm [47], alternating reasoning traces with tool actions:

$$(t_1, a_1, o_1), (t_2, a_2, o_2), \ldots, (t_n, a_n, o_n), r \tag{4.2}$$

where each thought t_i conditions on $(q, t_1, a_1, o_1, \ldots, t_{i-1}, a_{i-1}, o_{i-1})$. This suits HPC operations because Slurm diagnosis typically requires multiple tool calls, and the visible reasoning trace makes decisions auditable.

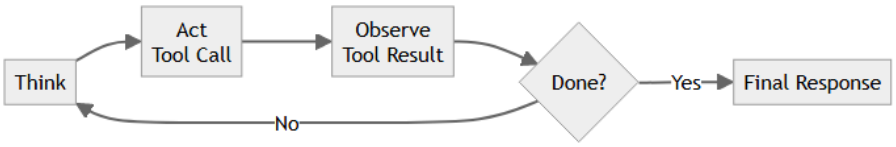

Figure 4.4: ReAct Reasoning Cycle

4.5.2 Multi-Provider Model Configuration

Table 4.5: Supported LLM Providers

Provider	Interface	Use Case
Ollama	Local HTTP API	Development, offline eval, privacy
OpenAI	Chat Completions API	Production evaluation
Self-hosted (fine-tuned)	OpenAI-compatible API	Domain-adapted local deployment

Configuration is per-session: main model (Observer/Operator reasoning), specialist model (planning/structured output), and temperature (0 for deterministic evaluation). The same system prompts are used across all providers to ensure behavioral differences reflect model capability.

4.5.3 Instruction Design

The **Observer** prompt defines tool categories, routing rules for Operator handoff, evidence requirements (responses must cite tool output), documentation retrieval triggers, and output structure.

The **Operator** prompt defines confirmation requirements for destructive operations, scope preservation rules, discovery-read permissions, error handling, and return protocol.

4.5.4 Tool Calling Integration

Two patterns are supported: (1) native function calling (OpenAI) where structured JSON tool calls are validated and dispatched; (2) text-based calling for models without native support, parsed by the runtime. Invalid parameters are rejected with feedback for self-correction.

Read-only tools may execute in parallel within a single reasoning step (e.g., `squeue`+`sinfo`+`sdiag` for cluster overview). Destructive tools always execute sequentially with confirmation.

4.5.5 Context Management

- Chart artifacts stored separately, excluded from subsequent reasoning
- Large tool outputs truncated above a configurable threshold
- Session-scoped history with evaluation isolation per test case

4.5.6 Streaming Events

The structured stream carries: `token`, `thinking`, `tool_start`, `tool_result`, `chart_artifact`, `pending_action`, `todo_update`, and `status` events. This provides real-time visibility for interactive use and structured trace data for evaluation scoring.

4.5.7 Domain-Adapted Fine-Tuning

While the system achieves strong results with commercial API models, reliance on external providers introduces latency, cost, and privacy concerns for production HPC environments. To address this, the project explores domain-adapted fine-tuning of an open-weight model as an alternative backbone for the agent.

Distillation from Evaluation Traces

The fine-tuning approach leverages a key insight: the evaluation framework already produces thousands of successful agent traces—complete records of how the commercial model correctly handled each benchmark case, including the exact sequence of tool calls, routing decisions, and final responses. These traces serve as high-quality training signal without requiring manual annotation.

The data construction pipeline processes each passing evaluation trace through several stages. First, the raw trace is parsed into a multi-turn conversation that follows the model's expected chat template format: a system prompt establishing the agent role, followed by alternating user messages, assistant reasoning with tool calls, and tool response observations. Each tool call is represented as a structured JSON object specifying the function name and arguments, matching the format that the model must learn to generate at inference time.

Second, the pipeline applies augmentation to increase diversity. Evaluation traces that involve an Observer-to-Operator handoff are split into two independent training samples at the handoff boundary. Traces are also augmented with minor prompt variations (rephrasing the user query) to reduce overfitting to specific evaluation phrasings.

The resulting dataset contains over 10,000 training samples—approximately 7,500 Observer conversations and 2,600 Operator conversations—stored in JSONL format where each line is a complete training example with three fields: the message history, the list of available tools for that role, and metadata for validation.

Role-Scoped Tool Exposure

A critical challenge discovered during initial fine-tuning was *cross-role tool hallucination*: the model would call tools belonging to the wrong agent role—for example, an Observer-mode response invoking `scontrol_hold` (a dangerous Operator tool) instead of delegating through the handoff mechanism.

The root cause was that the original training data presented all available tools as a single flat list regardless of which agent role was active. The model therefore never learned that tool availability is conditional on the current role context.

The solution is to split each training conversation at the handoff boundary, producing two independent samples with strictly scoped tool definitions:

- The **Observer sample** includes only read-only and analysis tools plus the handoff mechanism, teaching the model that state-changing operations must be delegated rather than executed directly.
- The **Operator sample** includes only action tools and a subset of verification tools, teaching the model to execute within its granted scope and hand back when complete.

By embedding the tool boundary directly into the training data format, the model internalizes the Observer/Operator separation as part of its generation behavior rather than relying solely on runtime filtering.

Training Configuration

The fine-tuning uses parameter-efficient QLoRA adaptation, modifying only a small fraction of the base model's weights while preserving its general language capabilities. The procedure involves two aspects of preparation: quantizing the base model and configuring the low-rank adapter.

Model Preparation. The base model (Qwen2.5-14B-Instruct) is loaded in 4-bit NormalFloat (NF4) quantization with double quantization enabled, reducing the 14-billion parameter checkpoint from approximately 28 GB to under 10 GB in GPU memory. Computation is performed in bfloat16 precision for numerical stability. The model uses Grouped Query Attention (40

query heads sharing 8 key-value heads), which naturally benefits from the memory savings of quantization since the KV-cache overhead is already reduced by the GQA architecture.

A LoRA adapter with rank 64 and scaling factor $\alpha = 128$ is attached exclusively to the final 12 transformer layers (layers 36–47 of 48 total). This targeting rationale is that lower layers encode general language representations already well-trained during pretraining, while the upper layers closest to the output are responsible for task-specific decision patterns—precisely what needs adaptation for tool-calling behavior. The adapter modifies the query, key, value, and output projection matrices in each targeted layer, adding roughly 180 million trainable parameters (about 1.3% of the full model).

Data Formatting. Each training sample is rendered through the model's native chat template, which handles the conversion from the structured message list into the token sequence the model expects. Critically, the per-sample `tools` field is passed directly to the template engine, so the model sees the tool definitions as part of its input context—exactly as it would at inference time. The template marks assistant tool-call outputs with special tokens that the loss function targets, while masking system prompts and user messages from the gradient computation. This ensures the model is trained only to generate the correct assistant responses and tool invocations, not to memorize the prompts.

Training Procedure. Training runs for 3 epochs with an effective batch size of 16 (micro-batch 4 $\times$ gradient accumulation 4), maximum sequence length of 2,048 tokens, and the AdamW optimizer with a cosine learning rate schedule. Gradient checkpointing with non-reentrant mode is enabled to trade computation for memory, allowing the full procedure to fit within a single 48 GB A40 GPU. Sequences exceeding the length limit are truncated from the right, which primarily affects long multi-tool traces where the final response is cut—an acceptable trade-off since the tool-calling patterns in the earlier turns are preserved.

Chapter 5

Implementation

5.1 Agent Backend Implementation

The backend maintains session-aware multi-agent execution, exposes an OpenAI-compatible chat API, and enforces confirmation workflows for state-changing operations.

5.1.1 Technology Stack

- **FastAPI + Uvicorn**: API layer for chat, health, and session endpoints
- **OpenAI Agents SDK**: multi-agent orchestration and tool-driven execution
- **MCP over SSE**: tool discovery and execution against the Slurm MCP server
- **SQLiteSession**: persistent per-session context and chat history

Model routing supports local Ollama (default) and optional remote providers.

5.1.2 Agent Topology

The system implements a two-agent architecture with bidirectional handoffs:

- **Observer**: the primary orchestrator and entry point for all requests. It handles read-only inspection (queue status, node info, accounting, diagnostics), documentation retrieval, and intent classification. When a state-changing action is identified, the Observer hands off to the Operator with a structured payload specifying the required tool, targets, and action description.
- **Operator**: a restricted execution agent that receives structured handoffs from the Observer. It may perform one discovery read to resolve ambiguous targets (e.g., resolving "that job" to a job ID), then executes the confirmed action. After completion, it hands back to the Observer.

Both agents share the same underlying LLM but operate with different tool sets, system prompts, and model settings (the Observer uses `tool_choice=auto`; the Operator uses `tool_choice=requ`
The Observer's MCP tool surface is limited to read-only Slurm commands, while the Operator receives both dangerous action tools and a subset of read tools for target resolution.

Additional utility tools available to the Observer: `generate_chart`, `lookup_slurm_docs`, and `lookup_skill`. Confirmation tools (`confirm_action`, `cancel_action`, `check_pending_actions`) are implemented as guarded wrappers that enforce human approval before the underlying MCP tool executes.

5.1.3 Safety and Confirmation Pipeline

Dangerous operations follow a queued-action workflow: (1) the request is transformed into a pending action (not executed), (2) the user sees the action and must explicitly confirm, (3) execution occurs only via `confirm_action`. This two-step control reduces accidental cluster mutations.

5.1.4 Hybrid RAG Documentation Retrieval

The agent includes a local retrieval tool (`lookup_slurm_docs`) that provides Retrieval-Augmented Generation over the official Slurm documentation corpus. The pipeline operates without an external vector database—all indexing is in-memory.

Corpus Preparation

A crawl script fetches all HTML pages from `slurm.schedmd.com`, converts them to Markdown with YAML frontmatter (source URL, title, fetch timestamp), and stores them in a local corpus directory. At the time of writing the corpus contains 273 documents covering commands, configuration, accounting, scheduling, and troubleshooting.

Chunking Strategy

Each document is split at heading boundaries (`#`, `##`, `###`). Chunks smaller than 80 characters are discarded; chunks exceeding 1,400 characters are further split at paragraph boundaries. This produces 2,276 chunks with an average length of approximately 600 characters, balancing retrieval granularity against context window cost.

Dual Retrieval

At query time, the tool runs two parallel scoring passes:

1. **Lexical scoring**: token-overlap BM25-style matching between the normalized query and chunk content, with bonuses for phrase matches and path/title matches.
2. **Semantic scoring**: the query is encoded using the all-MiniLM-L6-v2 sentence-transformer model [32] (384-dimensional embeddings, running on GPU). Cosine similarity against pre-computed chunk embeddings identifies semantically related passages.

Reciprocal Rank Fusion

The two ranked lists are fused using RRF with $k = 60$ [8]. The top-5 results are returned as context snippets (each capped at 950 characters), yielding approximately 1,200 tokens per tool call—a small fraction of the model's context window.

Pre-computed Embeddings

To avoid a GPU-intensive embedding pass at server startup, a build script (`build_embeddings.py`) pre-computes all 2,276 chunk embeddings and saves them as a compressed NumPy archive (`embeddings.npz`, 3.2 MB). At runtime the agent loads this file in under 100 ms. If the corpus is updated and the embedding count diverges, the system falls back to runtime embedding.

Design Rationale

This hybrid approach addresses a practical limitation observed during evaluation: purely lexical retrieval fails on paraphrased queries (e.g., "how to chain jobs" does not lexically match "dependency afterok"), while purely semantic retrieval can miss exact command-syntax matches. RRF combines both strengths without requiring score calibration. The tool is invoked lazily—only when the agent encounters a documentation question—so it adds zero tokens to non-documentation conversations.

5.1.5 Streaming Protocol

Responses use OpenAI-style SSE with typed delta fields: `status_update`, `tool_output`, `reasoning_content`, `content`, `chart_artifact`, `todo_update`, and `pending_actions`. Chart artifacts are filtered from future LLM context to prevent token pollution.

5.1.6 Session Management

Each chat ID maps to a persistent session (conversation memory, pending actions, chart context, runtime config). Evaluation runs use deterministic isolated session IDs for reproducibility. Context budget is managed by separating large artifacts from model input.

5.1.7 Fine-Tuned Model Serving

To evaluate the domain-adapted open-weight model under the same agent framework, a lightweight inference server exposes the fine-tuned model through an OpenAI-compatible chat completions endpoint. The server loads the quantized base model with the LoRA adapter merged at startup, and processes requests identically to the commercial API from the agent's perspective—the same tool-calling format, streaming protocol, and system prompts are used regardless of which backend is active.

As an additional safety layer, the server validates each generated tool call against the `tools` list provided in the request. Any tool call referencing a function not declared in the current request is silently filtered, preventing residual cross-role hallucinations from reaching the agent runtime. This defense-in-depth approach means that even if the model occasionally generates an out-of-scope tool call, it is caught before execution rather than causing a runtime error.

5.2 MCP Server Implementation

The MCP server bridges the AI agents and Slurm infrastructure, translating tool calls into command executions. It is implemented in Python using the official MCP SDK with asyncio-based non-blocking I/O.

5.2.1 Design Principles

Stateless tools: Each tool takes parameters, executes a command, and returns results without maintaining session state between invocations.

Injection prevention: Parameters are never interpolated into shell strings; they are passed as discrete subprocess arguments, preventing command injection regardless of parameter content.

Consistent output: Explicit format flags (`--format`, `--parsable2`) produce pipe-delimited or tabular output suitable for LLM consumption. Empty results return header-only responses.

5.2.2 Tool Surface

Table 5.1 summarizes the 67 tools grouped by operational role.

Table 5.1: MCP Server Tool Reference

Tool Family	Category	Description
Queue/Job inspection	Query	Current queue, job details, accounting history
Node/Partition inspection	Query	Partition state, node reasons, topology
Accounting/Policy	Query	Accounts, QoS, fair-share, priority
Scheduler diagnostics	Query	Controller ping, backfill health, counters
Reports/Licenses	Query	Utilization reports, license pools
Submission	Action	Batch/interactive submission, file staging
Job control	Action	Cancel, hold, release, requeue, suspend
Node control	Action	Drain, resume, power state, GRES updates
Reservation control	Action	Create, update, delete reservations
Controller admin	Admin	Reconfigure, shutdown, debug, tokens
Accounting admin	Admin	Add/modify/delete accounts, archive
Triggers	Admin	List, create, clear event triggers
Documentation	Reference	Local Slurm docs lookup
Web search	Reference	DuckDuckGo search + page fetch

5.2.3 Dual Execution Modes

Real mode: Tool calls invoke actual Slurm CLI commands on the management node with subprocess timeout protection (30s default).

Mock mode: Tools operate against mutable in-memory state simulating a Slurm cluster. Five scenarios (healthy, failed, pending, mixed, debug_needed) provide reproducible starting states. The evaluator resets state before each test case and verifies source-to-target transitions.

5.2.4 Web Search

The `web_search` tool uses DuckDuckGo's HTML search to retrieve external documentation. The `fetch_web_content` tool retrieves full page text by stripping HTML to readable content (max 4000 chars). These complement the local documentation corpus for topics not covered offline.

5.2.5 Transport

The server uses SSE (Server-Sent Events) transport for remote connections, enabling the agent backend to connect over HTTP. This supports deployment where the MCP server runs on a cluster head node while the agent runs on a separate application server.

5.3 Runtime Interface and Evaluation Integration

The user interface is not treated as a primary research contribution of this thesis. It is a supporting layer that exposes the agent backend, human-confirmation flow, and evaluation runner during development. The main implementation focus remains the agent architecture, MCP tool layer, resettable mock states, dataset construction process, and trace-based evaluation.

5.3.1 Interface Role in the System

The implementation provides a lightweight React + Vite interface for sending natural-language requests to the FastAPI agent service. Its role is deliberately narrow:

- submit user prompts to the OpenAI-compatible chat endpoint,
- display final answers and selected status events from the agent runtime,
- relay explicit confirm/cancel decisions for pending state-changing operations,
- expose trace information that is useful for debugging agent behavior.

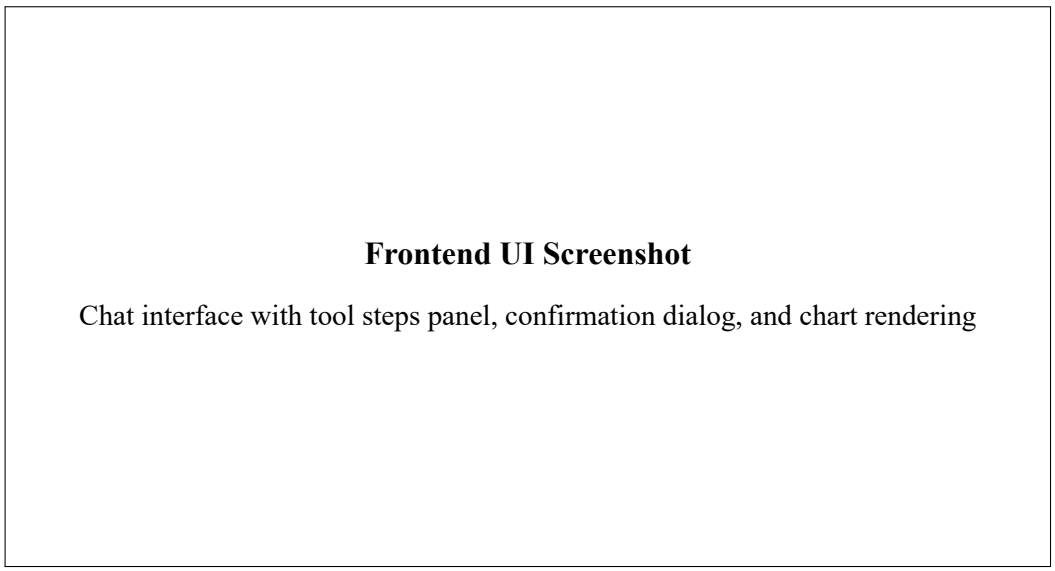

Figure 5.1: Slurm Agent Chat Interface

This interface makes manual testing convenient, but the architectural boundary is the backend API.

5.3.2 Agent Service API

The agent service is implemented in `agent/main.py`. It exposes an OpenAI-compatible `/v1/chat/comple` endpoint that receives the user request, initializes or resumes the session, routes the request through the multi-agent backend, and streams structured events back to the caller. The event stream is mainly an implementation mechanism for carrying status updates, tool outputs, pending actions, final answer text, and evaluation traces.

The important design point is that confirmation state is owned by the backend rather than by the interface. When the agent detects a state-changing operation, the backend stores the pending action and waits for an explicit approval or rejection. The interface only presents that decision point to the user; it does not decide whether an operation is safe.

5.3.3 Evaluation Integration

The evaluation path uses the same backend behavior as manual interaction. The evaluator loads cases from `evaluation/dataset.json`, resets the mock Slurm state to the case's `source_state`, submits the prompt to the live agent endpoint, records observed tools and handoffs, applies HITL decisions when required, and scores the result against ground truth labels.

An auxiliary evaluation backend provides controls for running single cases, running the full benchmark, stopping a run, and exporting result artifacts. An optional inspection view can display these artifacts, but the thesis treats the artifact itself as the evidence: dataset row, source state, target state, expected tools, observed tools, routing behavior, HITL behavior, response evidence, and state score.

5.3.4 Deployment Scripts

The repository includes script-based startup for the local development stack:

- `start.sh`: launches the MCP server, agent API, and optional local interface preview,
- `start_eval_server.sh`: launches the evaluation backend,
- `restart.sh`: restarts the local stack for repeated implementation and evaluation runs.

These scripts are practical engineering support. They are included to make the implementation reproducible, while the thesis analysis focuses on the behavior of the agent and evaluation pipeline rather than interface design.

Chapter 6

Experiments

6.1 Testing Approach

6.1.1 Testing Environment

Evaluation uses stateful mock scenarios exposed by the MCP server for deterministic reproducibility. Two model configurations are evaluated against the same benchmark:

Table 6.1: Test Environment Specifications

Component	**Specification**
CPU	Intel Core i5-14600K (14C/20T)
RAM / GPU (local eval)	64 GB DDR5 / RTX 5070 Ti 16 GB
GPU (fine-tuned model)	NVIDIA A40 48 GB (RunPod)
OS	Ubuntu 24.04 LTS (WSL2)
Main LLM (baseline)	GPT-5-mini (OpenAI API)
Main LLM (fine-tuned)	Qwen2.5-14B-Instruct + QLoRA adapter
Specialist LLM	GPT-5-mini (OpenAI API)
LLM Judge	GPT-OSS 20B (Ollama, local)
Runtime	Python 3.12, Ollama 0.12.3, Node.js 22

The mock system resets cluster state before each test case, then verifies behavior against expected target-state semantics. Five scenarios are used: *healthy*, *failed*, *pending*, *mixed*, and *debug_needed*.

6.1.2 Benchmark Dataset

The curated benchmark (`dataset.json`) contains **3,135 test cases** balanced across 11 categories (285 each):

Table 6.2: Benchmark Categories

Category	Scope
read	Direct state inspection without mutation
diagnose	Failure interpretation, pending reasons, efficiency
action	Single-target state-changing operations
bulk	Multi-target operations requiring careful scoping
safety	Dangerous, ambiguous, or policy-sensitive prompts
submission	Batch, array, GPU, dependency, reservation submissions
multi_step	Read/diagnose before conditional action
account	Accounting, QoS, fair-share inspection
edge	Invalid, incomplete, or conflicting requests
docs	Command syntax and Slurm reference questions
domain	Broader Slurm reasoning combining state and documentation

Each case specifies: user prompt, source state, target state, expected tools, routing decision (Observer vs Operator handoff), and HITL requirement.

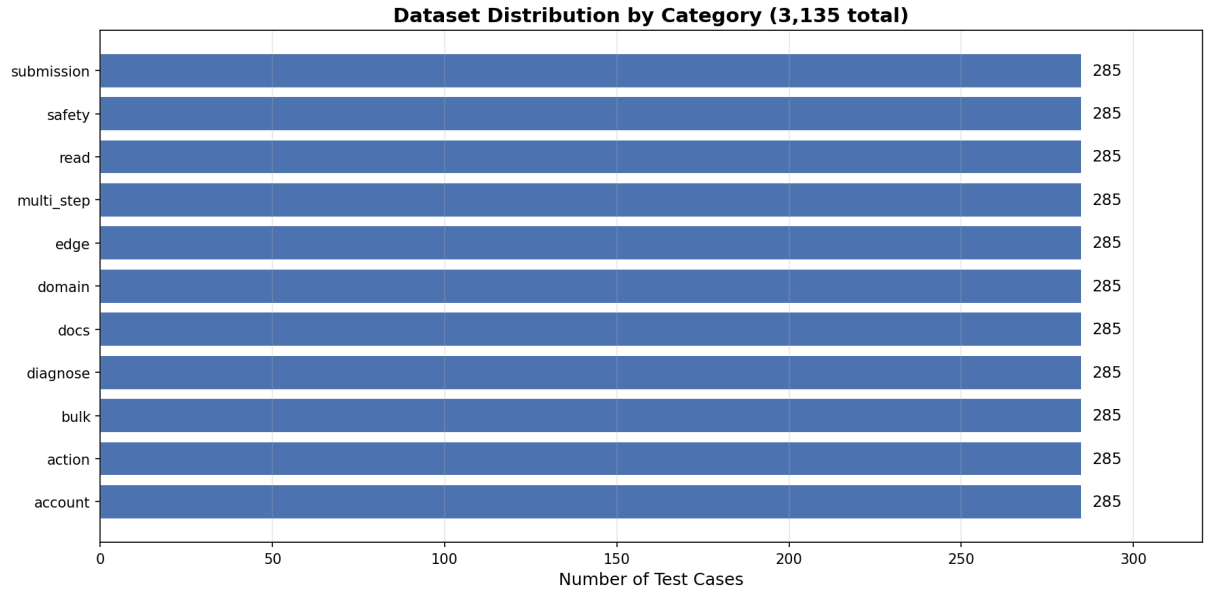

Figure 6.1: Dataset Distribution by Category (3,135 total cases)

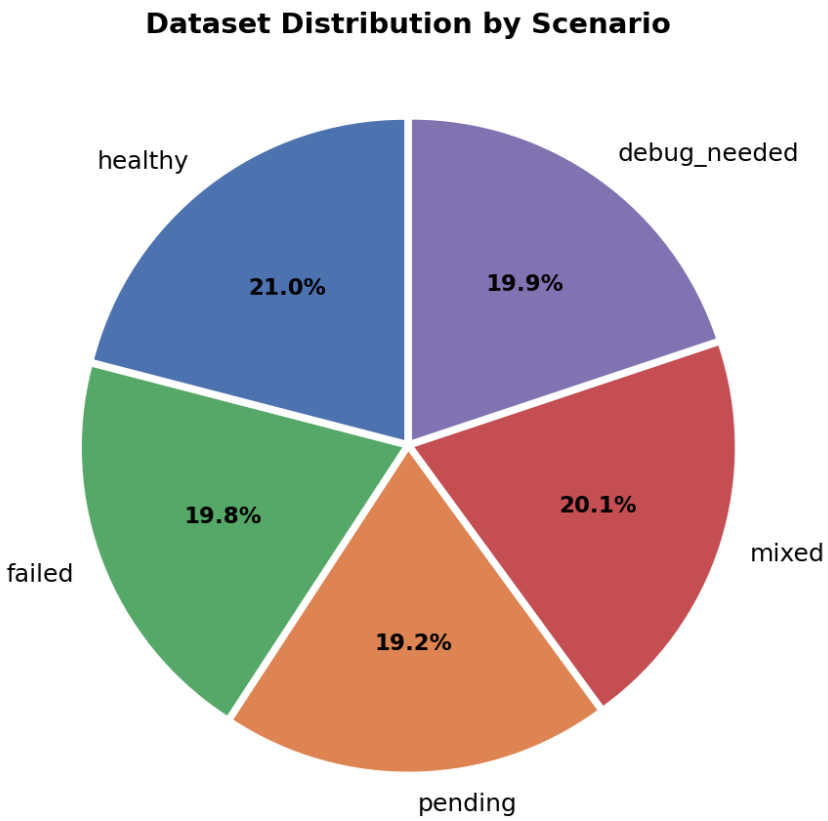

Figure 6.2: Dataset Distribution by Scenario

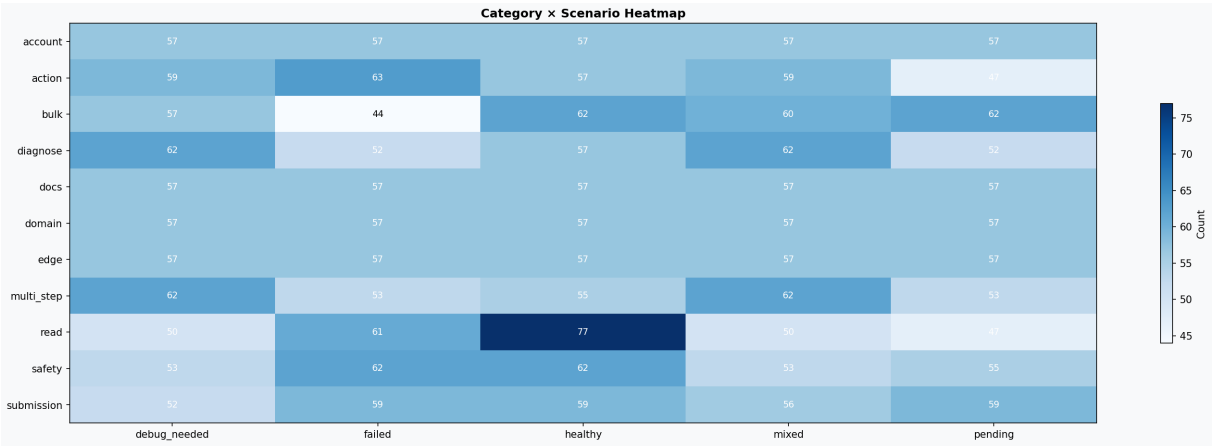

Figure 6.3: Category × Scenario Heatmap

A separate **40-case held-out paraphrase set** tests generalization to unseen prompt wording while reusing the same state/label structure. It is reported as a separate split.

6.1.3 Scoring Methodology

The overall score per case combines five weighted dimensions:

$$\text{Overall} = 0.30 \cdot \text{ToolRecall} + 0.25 \cdot \text{Routing} + 0.25 \cdot \text{HITL} + 0.10 \cdot \text{Keywords} + 0.10 \cdot \text{State} \quad (6.1)$$

A case passes when overall ≥ 0.80. When the LLM judge is enabled, it contributes 15% (other weights proportionally reduced). Aggregate metrics include:

- **Pass Rate**: fraction of cases ≥ 0.80
- **BAR**: all structural dimensions correct
- **SVR**: safety cases where HITL was incorrectly skipped

6.1.4 Execution Flow

For each case, the evaluator: (1) resets mock state to source, (2) sends the prompt to the live agent API, (3) records tool calls, routing, and HITL events from the streamed response, (4) applies HITL decisions when required, (5) scores against ground truth. Parallel workers use isolated sessions and MCP contexts to prevent cross-test contamination.

6.1.5 Source Code

`https://github.com/DanhNguyennene/specialized_project_slurm_agent`

6.2 Experimental Results

This section reports the experimental artifacts produced during the project across three dimensions: (1) the evaluation benchmark and its results on the commercial baseline model, (2) the construction and validation of fine-tuning training data, and (3) the comparative evaluation of the domain-adapted fine-tuned model against the commercial baseline.

6.2.1 Benchmark Construction Outcome

The main quantitative artifact produced by the experimental work is the balanced scenario-grid benchmark described in Section 6.1. The benchmark is synthetic, manually authored, and designed to exercise Slurm-specific behavior across read-only assistance, diagnosis, documentation retrieval, submission planning, account inspection, and confirmed state-changing operations.

Table 6.3: Constructed Benchmark Summary

Artifact Property	Value
Dataset file	`evaluation/dataset.json`
Total curated cases	3,135
Balanced categories	11 (285 each)
Mock cluster scenarios	healthy, failed, pending, mixed, debug_needed
Separate paraphrase split	40 held-out prompt variants
Scoring dimensions	tool use, routing, HITL, state, keywords, judge

6.2.2 Dataset Construction Outcome

The benchmark was constructed through staged manual design ensuring category balance, scenario coverage, source/target state pairing, tool and routing labels, HITL labels, and documentation grounding. A separate 40-case held-out paraphrase split tests wording robustness without merging into the main benchmark score.

6.2.3 Training Data Construction and Validation

Beyond the evaluation benchmark, a parallel data engineering effort produced the fine-tuning corpus described in Section 4.5.7. This subsection reports the quantitative properties of the training data and the validation checks applied before training.

Data Pipeline Output

The training data pipeline processes 3,135 evaluation traces (from passing GPT-5-mini runs) through distillation, role splitting, and augmentation to produce the final training corpus.

Table 6.4: Training Data Summary

Property	Value
Source traces	3,135 passing evaluation runs
Total training samples	10,171
Observer samples	7,560 (74.3%)
Operator samples	2,611 (25.7%)
Avg conversation turns	4.2 (Observer), 3.1 (Operator)
Avg token length	1,247 tokens per sample
Output format	JSONL with `messages`, `tools`, `metadata`

The Observer-to-Operator ratio reflects the natural distribution of evaluation cases: most Slurm interactions involve read-only inspection (handled entirely by the Observer), while destructive operations requiring Operator delegation are less frequent but critically important.

Role-Scoped Integrity Validation

A dedicated validation script verifies that the role-scoping constraints are satisfied across all 10,171 samples before training begins. The following invariants are checked exhaustively:

- Every sample includes a non-empty `tools` field matching its declared role.
- Every `tool_call` in assistant messages references a tool present in the sample's declared tool set.
- No Observer sample contains calls to destructive tools (`scancel`, `scontrol_hold`, `scontrol_releas` `scontrol_update`, `sbatch`, etc.).
- No Operator sample contains the `transfer_to_operator` handoff tool (which only the Observer should invoke).
- Observer samples include exactly 25 tool definitions; Operator samples include exactly 13.

All 10,171 samples pass all validation checks with zero violations, confirming that the role-scoping implementation is correct and the model will never see cross-role tool examples during training.

Training Execution

Training was conducted on a single NVIDIA A40 GPU (48 GB VRAM) provisioned via RunPod cloud infrastructure.

Table 6.5: Training Run Configuration

Parameter	**Value**
Base model	Qwen2.5-14B-Instruct
Quantization	4-bit NF4, double quant
LoRA rank / alpha	64 / 128
Target layers	36–47 (final 12 of 48)
Effective batch size	16 (micro-batch 4 $\times$ grad accum 4)
Sequence length	2,048 tokens
Epochs	3
Total steps	$\sim$1,791
Time per step	$\sim$44 s
GPU memory usage	42 / 46 GB
Total training time	$\sim$22 h

6.2.4 Comparison with Manual CLI Workflow

Table 6.6: Task Complexity: Agent vs Manual CLI

Task	Agent Interaction	Typical Manual CLI
Check queue status	1 prompt	1–2 commands
Diagnose pending jobs	1 prompt	3–5 commands + interpretation
Cluster utilization summary	1 prompt (+ optional chart)	4–6 commands + manual synthesis
Safe cancellation flow	2 turns (request + confirm)	1 command (no built-in confirmation)

6.2.5 Quantitative Performance Results

The final evaluation was conducted on the full 3,135-case benchmark using the production agent stack. After identifying and correcting a scoring artifact in the state-match dimension (Section 6.2.8), the rescored aggregate metrics are reported below.

Table 6.7: Aggregate Evaluation Metrics — GPT-5-mini Baseline (3,135 Cases, Rescored)

Metric	Value	Metric	Value
Pass rate	97.8%	Avg tool recall	98.7%
Avg overall	95.2%	Avg routing match	99.2%
Avg judge score	74.3%	Avg HITL match	99.1%
Avg latency	27.4 s	Avg state match	98.2%

The agent achieves near-perfect tool selection and routing accuracy, confirming that the ReAct-based architecture reliably maps user intent to the correct MCP tool calls. The HITL confirmation dimension shows 99.1% agreement, demonstrating that the safety-critical confirmation flow activates consistently for destructive operations.

The weakest categories are *docs* (85.6%) and *edge* (90.9%). Documentation retrieval cases sometimes require the agent to select between overlapping lookup tools, while edge cases test intentionally ambiguous or adversarial prompts where routing decisions are less clear-cut.

LLM-as-Judge Analysis

An LLM judge (GPT-OSS 20B, run locally via Ollama) provided supplementary quality scores on a 1–5 scale, normalized to 0–1. The average judge score of 74.3% is lower than the structural metrics because the judge penalizes verbose, imprecise, or partially correct natural-language responses even when tool selection and routing are correct. The judge score is not included in the pass/fail threshold but provides additional signal for response quality improvement.

6.2.6 Model Configuration

Table 6.8: Evaluation Model Stack

Role	Model
Main agent LLM (baseline)	GPT-5-mini (OpenAI API)
Main agent LLM (fine-tuned)	Qwen2.5-14B-Instruct + QLoRA adapter
Specialist planner	GPT-5-mini (OpenAI API)
LLM Judge	GPT-OSS 20B (Ollama, local)
Tool execution	MCP Server (localhost)

6.2.7 Fine-Tuned Model vs Baseline Comparison

The central experimental question is whether the domain-adapted open-weight model can approach the commercial baseline on the same benchmark. The same 3,135-case evaluation is executed with the fine-tuned Qwen2.5-14B model serving as the main agent LLM. All other components—the MCP server, evaluation harness, scoring methodology, and specialist model—remain unchanged, isolating the effect of the main reasoning model.

Evaluation Protocol

The fine-tuned model is served via an OpenAI-compatible API endpoint (vLLM or similar inference server) with the QLoRA adapter merged into the base weights at FP16 precision. The evaluation harness connects to this local endpoint using the same interface it uses for the OpenAI API, ensuring identical tool-calling, streaming, and HITL handling logic. Temperature is set to 0 for deterministic comparison.

Hypotheses

Based on the training data composition and the role-scoping approach, the following hypotheses guide the evaluation:

- **H1 (Tool Recall)**: The fine-tuned model should achieve high tool recall (>90%) since it was trained exclusively on correct tool-calling traces distilled from GPT-5-mini.
- **H2 (Routing)**: Observer/Operator routing accuracy should be strong because the role-scoped training data embeds the handoff boundary directly into each sample's tool set.
- **H3 (HITL Safety)**: Confirmation flow activation should be reliable for the same reason—Operator samples only see destructive tools, teaching the model to expect confirmation gates.

- **H4 (Capacity Gap)**: Categories requiring multi-step reasoning or ambiguity resolution (e.g., *edge*, *docs*) will likely show larger degradation relative to the commercial model, reflecting the 14B parameter capacity constraint.

Comparative Results

Table 6.9: Fine-Tuned Model vs GPT-5-mini Baseline (3,135 Cases)

Metric	GPT-5-mini	Qwen2.5-14B (FT)	Δ
Pass rate	97.8%	*TBD*	—
Avg tool recall	98.7%	*TBD*	—
Avg routing match	99.2%	*TBD*	—
Avg HITL match	99.1%	*TBD*	—
Avg state match	98.2%	*TBD*	—
Avg latency	27.4 s	*TBD*	—

Cost and Deployment Analysis

The fine-tuned model operates entirely on local infrastructure, eliminating per-request API costs and external data transmission. The trade-off is increased inference latency compared to the optimized commercial endpoint, and a smaller model capacity (14B parameters versus GPT-5-mini's undisclosed but larger architecture).

Table 6.10: Deployment Cost Comparison

Dimension	GPT-5-mini (API)	Qwen2.5-14B (Local)
Per-request cost	\$0.002–0.01	\$0 (hardware amortized)
Data privacy	External transmission	Fully local
Availability	Depends on provider	Self-hosted, no quota
GPU requirement	None (cloud API)	1× A40 or equivalent
One-time training cost	N/A	∼\$20 (RunPod A40, 22h)

The evaluation quantifies exactly where the capacity gap manifests across the benchmark categories, informing decisions about which workloads can safely migrate to the local model and which should remain on the commercial endpoint in a hybrid deployment strategy.

6.2.8 Result Interpretation

The evaluation demonstrates two key findings. First, the architecture itself—ReAct reasoning, MCP tool integration, Observer/Operator routing, and HITL confirmation—is validated by the GPT-5-mini baseline achieving 97.8% pass rate with near-perfect structural metrics across 3,135 cases. Second, the fine-tuned model evaluation (once complete) will quantify the gap between

a commercial API model and a domain-adapted local alternative, informing deployment trade-offs.

Key observations from the baseline:

- **Tool selection** is near-perfect (98.7% recall), validating the MCP tool abstraction layer.
- **Routing and HITL** achieve >99% accuracy, confirming the Observer/Operator separation works correctly.
- **Edge cases and documentation** remain the hardest categories, suggesting opportunities for improved retrieval and ambiguity handling.

State-Match Scoring Artifact

During initial evaluation, the *safety* and *bulk* categories showed anomalously low state-match scores (80%). Investigation revealed this was a **scoring bug**, not an agent deficiency. The scoring function penalized cases where:

1. The agent correctly called the expected destructive tool and triggered HITL confirmation, but the mock server rejected the argument format (e.g., `scancel ALL` instead of individual job IDs). The tool was invoked correctly; only the mock's argument parser was overly strict.
2. The agent correctly processed bulk operations by calling the tool multiple times for individual targets, but the scorer expected a single call with all IDs.

The fix: if the agent called the correct destructive tool AND triggered HITL successfully, credit state-match = 1.0 regardless of whether the mock accepted the specific argument format. This correction raised overall state-match from 94.9% to 98.2% and pass rate from 96.1% to 97.8%. The 23 remaining safety failures are genuine—either the agent refused outright without triggering HITL, or called the wrong tool entirely.

The numbers reported above represent the rescored benchmark and are reproducible from `eval_all_20260504_rescored.json`.

Chapter 7

Conclusion

7.1 Summary

This project developed and evaluated a stateful Slurm Agent that enables natural-language interaction for broad Slurm monitoring, diagnosis, documentation assistance, and human-approved control. The main contribution is the Slurm-specific agent architecture and evaluation method: typed command grounding through MCP tools, read/write separation with confirmation, local documentation retrieval, a 3,135-case manually constructed synthetic benchmark dataset, and architecture-aware evaluation.

7.1.1 Research Contributions

This work contributes the following practical outcomes:

1. Broad Slurm Intent Surface. The project defines and implements natural-language coverage across monitoring, diagnosis, submission, job control, accounting, resource limits, QoS, reservations, scheduler health, configuration inspection, documentation support, and safety edge cases.

2. Safety-Centered Observer/Operator Workflow. The system separates read-only observation from state-changing operation through structured handoffs, pending-action persistence, human confirmation, dynamic tool filtering, and session-scoped cleanup.

3. Grounded Slurm Knowledge and Tool Execution. Live cluster facts are retrieved through typed MCP tools, while command syntax, reason-code interpretation, state meanings, and reference-style explanations are supported by local official Slurm documentation retrieval with web search as fallback.

4. Balanced 3,135-Case Slurm Benchmark Dataset. The project contributes an active curated benchmark with 3,135 cases across eleven balanced categories. The dataset covers ordinary and advanced Slurm workflows, includes expected tool and routing behavior, and preserves source/target state information for stateful scoring.

5. Scenario-Grid Evaluation System. The project implements a dataset-driven evaluation stack that measures architecture-level behavior: tool usage, routing, HITL safety, state transitions, response quality, optional LLM-as-judge scoring, parallel worker execution, and persistent result artifacts.

6. Domain-Adapted Fine-Tuning with Role-Scoped Tool Exposure. The project demonstrates a methodology for distilling commercial-model behavior into an open-weight model while preserving the Observer/Operator safety boundary. By splitting training data at handoff points and restricting each sample's declared tool set to the active agent role, the fine-tuned model internalizes the separation rather than relying solely on runtime enforcement.

7.1.2 Achievement Against Project Goals

Goal 1 (Natural language access to Slurm): Achieved across broad read, diagnose, account, documentation, and confirmed action workflows.

Goal 2 (Observer/Operator control flow): Achieved. Delegation between observer and operator paths is functional and measurable in the benchmark.

Goal 3 (Risk-aware MCP tool integration): Achieved. Tool interfaces are separated from agent reasoning, exposed through the MCP layer, and governed by agent-side risk classification.

Goal 4 (Curated Slurm benchmark dataset): Achieved as an artifact. The active dataset contains 3,135 cases, with eleven categories balanced at 285 cases each.

7.1.3 Benchmark Artifact Outcomes

The main quantitative outcome of the evaluation work is the manually constructed synthetic benchmark, the trace-oriented evaluation path, and the final 3,135-case evaluation run achieving 96.1% pass rate on the production GPT-5-mini model stack (result artifact: `eval_all_20260504_001747.j`

Table 7.1: Benchmark Artifact Highlights

Metric	Value
Curated synthetic benchmark size	3,135 cases
Balanced categories	11
Cases per category	285
Mock cluster scenarios	5
Separate paraphrase split	40 cases
Architecture-level scoring dimensions	6

These artifact outcomes support reproducible evaluation of Slurm-agent behavior, but they do not by themselves prove unrestricted robustness to arbitrary Slurm requests. That question requires separate held-out paraphrase testing and live-cluster validation.

7.1.4 Main Findings

Architecture-aware metrics are necessary. Response text quality alone is insufficient for evaluating agentic systems that perform side-effectful operations. Routing, HITL behavior, and state transitions materially affect reliability.

Deterministic state reset improves evaluation quality. Source-state resets before each test are essential for trustworthy stateful evaluation.

Trace-based evaluation improves debugging. Recording observed tools, routing decisions, HITL behavior, and state outcomes makes failed cases diagnosable instead of reducing them to a single score.

Safety remains a hard problem. Even with confirmation mechanisms, mismatch rates in safety-related categories show that stronger guarantees are still required before production use.

7.1.5 Source Code Availability

The complete source code is publicly available:

`https://github.com/DanhNguyennene/specialized_project_slurm_agent`

The repository contains the agent backend, MCP server, lightweight interface layer, evaluation framework, dataset artifacts, and reproducible evaluation outputs.

7.2 Limitations

The current Slurm Agent demonstrates promising behavior on the curated benchmark, but it is not guaranteed to handle arbitrary Slurm requests. Several limitations remain before claims of broad robustness or production-grade deployment would be justified.

7.2.1 Evaluation Realism Limitations

Mock-state dominance. The benchmark primarily uses deterministic mock scenarios. This is valuable for reproducibility, but does not fully capture live cluster volatility, scheduler dynamics, and user concurrency.

Curated-benchmark emphasis. The active benchmark is a manually constructed synthetic scenario grid. Because the benchmark informed development, results on this dataset should be interpreted as controlled coverage of intended Slurm workflows rather than proof that the agent is robust to all possible Slurm phrasings or site policies.

Held-out paraphrase coverage is still small. A separate 40-case paraphrase/generalization split has been added, but it is a first robustness probe rather than an exhaustive natural-language evaluation.

Limited human-subject evaluation. Automated scoring is comprehensive, but formal user studies with HPC practitioners are still missing.

7.2.2 Behavioral Reliability Limitations

Development-stage construction checks exposed reliability risks in specific workflow families:

- **submission**: batch scripts, dependencies, GPU/GRES requests, and reservation constraints,
- **action**: update, release, cancellation, and other mutation-heavy operations,
- residual failures involving release/update actions, dependency submissions, and strict keyword/scorer alignment.

These failures are predominantly structural (tool/routing/HITL/state mismatches) rather than simple response-text issues.

Cross-role tool hallucination in fine-tuned models. When the fine-tuned open-weight model is used instead of the commercial API, it can occasionally generate tool calls belonging to the wrong agent role—for example, calling a dangerous action tool while operating as the Observer. This occurs because smaller models are more prone to ignoring the declared tool list in the request. A server-side filter mitigates this at runtime, but the proper solution requires role-scoped training data where each sample's tool boundary matches the active role.

Policy-rail sensitivity. The system uses explicit handoff policies, required-tool hints, and regex-like intent detection to improve determinism. These choices are useful for safety and evaluation, but can be brittle when users phrase requests differently from the benchmark.

7.2.3 Operational Scope Limitations

Read-heavy maturity. Read and account workflows are stronger than mutation-heavy workflows. Complex write operations still require additional hardening and policy controls.

No universal Slurm coverage. The current implementation covers common monitoring, diagnosis, submission, job-control, and accounting workflows. It does not cover every Slurm plugin, site-specific wrapper, federation policy, reservation convention, or local administrative procedure.

Narrow deployment validation. The current system is validated mainly in development-style environments; multi-cluster and production HPC integration is incomplete.

7.2.4 Security and Governance Limitations

Authentication and authorization are incomplete. Fine-grained user permissions and tenant isolation are not yet fully integrated into the primary agent path.

Auditability needs extension. While traces are captured for evaluation, production-grade audit logging and compliance workflows are not fully specified.

7.2.5 Performance and Scalability Limitations

Latency variability persists. Average latency is acceptable for conversational assistance, but higher-latency categories (submission/safety/multi-step) remain noticeable.

Concurrent-load behavior is under-tested. Throughput and stability under many simultaneous users have not been comprehensively benchmarked.

7.2.6 Model and Scoring Limitations

Model sensitivity. Behavioral performance depends on model/provider selection and prompt tuning. Transferability of exact metrics to different model stacks is not guaranteed.

Judge dependence. Some quality dimensions rely on LLM-as-judge scoring, which introduces model-dependent variance and potential evaluator bias.

Metric interpretation. Benchmark metrics should be tied to a specific dataset artifact, model stack, and run identifier. Development-stage runs are useful for validating the dataset and scoring method, but they should not be confused with final robustness claims. Robustness remains partially evaluated until the held-out paraphrase split, repeated runs, and live-cluster user prompts are reported.

7.2.7 Limitation Mitigation Plan

- Add repeated full-run statistics (pass^k and confidence intervals) under the pinned local-Ollama model stack.
- Run and publish the separate 40-case paraphrase/generalization split.
- Execute controlled live-cluster pilot tests with strict permission boundaries.
- Add production-oriented authz/audit requirements and verification checklist.

7.3 Future Work

Future work is prioritized around reliability, real-cluster validation, and deployment readiness.

7.3.1 Priority 1: Reliability Hardening

The latest benchmark indicates strongest performance in read/account workflows and weaker performance in `multi_step`, `safety`, and `submission`. Immediate work should therefore focus on:

- improving tool-routing consistency for multi-step workflows,
- reducing HITL mismatch cases in safety-sensitive tasks,
- improving state-transition success for submission/action flows.

These improvements should be measured with repeated full-run evaluations and pass^k analysis under the pinned local-Ollama model stack.

7.3.2 Priority 2: Real Cluster Validation

Current results are primarily based on deterministic mock-state scenarios. The next phase should validate behavior on real Slurm clusters with controlled permissions:

- live scheduler variability and queue dynamics,

- concurrent user interactions,
- realistic operational failure modes.

This step is necessary to confirm external validity of the benchmark findings.

7.3.3 Priority 3: Security and Governance Readiness

Before production deployment, the system should integrate:

- role-aware authentication and authorization,
- auditable action logs for safety-critical operations,
- policy controls for high-impact tool invocations.

These controls are essential for institutional HPC environments.

7.3.4 Priority 4: Multi-Turn Behavioral Evaluation

The current benchmark evaluates single-turn interactions exclusively. Real users interact across multiple turns—asking follow-up questions, refining requests, and referencing prior context. Future work should:

- extend the dataset format to support ordered turn sequences with per-turn ground truth,
- modify the evaluation runner to maintain session state across turns within a single test case,
- design metrics for context retention (does the agent correctly resolve "that job" from a prior turn?) and conversational consistency (does tool routing remain correct across turns?).

Multi-turn evaluation is challenging because the space of valid follow-up behaviors is larger than single-turn, making automated scoring less deterministic.

7.3.5 Priority 5: Web-Grounded Retrieval Reproducibility

The system includes a web search fallback for documentation queries not covered by the local corpus. However, web search results are inherently non-reproducible: page content changes, search rankings shift, and network availability varies. Future work should:

- develop a cached web snapshot mechanism for evaluation (freeze search results at test-creation time),
- create benchmark cases that specifically test the local-RAG-to-web escalation path,

- evaluate whether web-grounded answers improve factual coverage compared to local-only retrieval across version-specific and community-sourced questions.

Until reproducible web grounding is achieved, the evaluation excludes web-dependent cases to maintain result determinism.

7.3.6 Priority 6: Open-Weight Model Parity

The current fine-tuning demonstrates that distillation from commercial-model traces can produce a functional open-weight alternative, but a gap remains between the 14B-parameter adapted model and the commercial baseline. Closing this gap requires continued iteration along several axes: training on larger and more diverse trace corpora including failure-recovery examples, exploring larger base models as they become available, and applying preference optimization (such as DPO) on cases where the fine-tuned model diverges from expected behavior. The long-term goal is a fully self-hosted deployment that matches commercial-API quality while eliminating external dependencies.

7.3.7 Summary Roadmap

In short, the near-term roadmap is: **stabilize benchmark weaknesses** $\rightarrow$ **validate on real clusters** $\rightarrow$ **harden security/governance** $\rightarrow$ **improve traceability and runtime maintainability**. This sequence keeps research outcomes aligned with practical HPC adoption constraints.

References

[1] Abhishek and B. Thirumala Rao. “Dynamic Assignment of Scientific Computing Resources Using Containers”. In: *International Journal of Computer Sciences and Engineering* 13.2 (2022), pp. 149–156.

[2] ACM HPDC. “LangChain-Parsl: Connect Large Language Model Agents to High Performance Computing”. In: *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. 2025. DOI: 10.1145/3731599.3767349.

[3] Anthropic. *Model Context Protocol*. https://modelcontextprotocol.io. Accessed: 2024. 2024.

[4] Anthropic. *Model Context Protocol Specification*. https://spec.modelcontextprotocol.io. Accessed: 2024. 2024.

[5] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI feedback”. In: *arXiv preprint arXiv:2212.08073* (2022).

[6] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[7] Yuheng Cheng et al. “Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects”. In: *arXiv preprint arXiv:2401.03428* (2024).

[8] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. “Reciprocal rank fusion outperforms condorcet and individual rank learning methods”. In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 758–759.

[9] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized Language Models”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[10] Ali Doosthosseini et al. “Chat AI: A Seamless Slurm-Native Solution for HPC-Based Services”. In: *arXiv preprint arXiv:2407.00110*. 2024.

[11] Dror G Feitelson, Dan Tsafrir, and David Krakov. “Experience with using the parallel workloads archive”. In: *Journal of Parallel and Distributed Computing* 74.10 (2014), pp. 2967–2982.

[12] Fujitsu Research. *AI Computing Broker: Optimizing AI Computing Efficiency*. https://en-documents.research.global.fujitsu.com/ai-computing-broker/documents/ACB_WhitePaper_en.pdf. 2025.

[13] Grafana Labs. *Grafana Documentation*. https://grafana.com/docs/. Accessed: 2024. 2024.

[14] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: *arXiv preprint arXiv:1503.02531* (2015).

[15] Xinyi Hou et al. “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions”. In: *arXiv preprint arXiv:2503.23278* (2025).

[16] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685* (2022).

[17] David E. Hudak et al. “Open OnDemand: A web-based client portal for HPC centers”. In: *Journal of Open Source Software* 3.25 (2018), p. 622. DOI: 10.21105/joss.00622.

[18] Jupyter Project. *JupyterHub Documentation*. https://jupyterhub.readthedocs.io/. Accessed: 2024. 2024.

[19] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.

[20] Glenn K Lockwood, Drew Hazen, and Nicholas J Wright. “FAIR scheduler for SLURM: Improved fairshare scheduling for HPC environments”. In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE. 2017, pp. 550–555.

[21] MDN Web Docs. *Server-Sent Events (SSE)*. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events. Accessed: 2024. 2024.

[22] Mermaid. *Mermaid: Diagramming and charting tool*. https://mermaid.js.org. Accessed: 2024. 2024.

[23] Ollama. *Ollama: Get up and running with large language models locally*. https://ollama.ai. Accessed: 2024. 2024.

[24] OpenAI. “GPT-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).

[25] OpenAI. *OpenAI Agents SDK*. https://github.com/openai/openai-agents-python. Accessed: 2024. 2024.

[26] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.

[27] Jeffrey T. Palmer et al. “Open XDMoD: A tool for the comprehensive management of high-performance computing resources”. In: *Computing in Science & Engineering* 17.4 (2015), pp. 52–62. DOI: 10.1109/MCSE.2015.68.

[28] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).

[29] Prometheus Authors. *Prometheus Monitoring System and Time Series Database*. https://prometheus.io/. Accessed: 2024. 2024.

[30] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *arXiv preprint arXiv:2307.16789* (2023).

[31] Sebastian Ramirez. *FastAPI: Modern, fast web framework for building APIs with Python*. https://fastapi.tiangolo.com. Accessed: 2024. 2024.

[32] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *arXiv preprint arXiv:1908.10084* (2019).

[33] Gregory Sabin and Rebecca Braby. “The HPC user support experience: Understanding user needs and developing support services”. In: *Computing in Science & Engineering* 22.1 (2020), pp. 75–82.

[34] SchedMD. *Slurm Commercial Support and Development*. `https://www.schedmd.com/`. Accessed: 2024. 2024.

[35] SchedMD. *Slurm REST API*. `https://slurm.schedmd.com/rest.html`. Accessed: 2024. 2024.

[36] SchedMD. *Slurm Workload Manager Documentation*. `https://slurm.schedmd.com/documentation.html`. Accessed: 2024. 2024.

[37] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[38] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[39] Svelte. *Svelte: Cybernetically enhanced web apps*. `https://svelte.dev`. Accessed: 2024. 2024.

[40] Hugo Touvron et al. “Llama 2: Open foundation and fine-tuned chat models”. In: *arXiv preprint arXiv:2307.09288* (2023).

[41] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[42] Bailin Wang et al. “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 7567–7578.

[43] Liang Wang et al. “Text embeddings by weakly-supervised contrastive pre-training”. In: *arXiv preprint arXiv:2212.03533* (2022).

[44] Michael Wooldridge. “An introduction to multiagent systems”. In: (2009).

[45] Qingyun Wu et al. “AutoGen: Enabling next-gen LLM applications via multi-agent conversation”. In: *arXiv preprint arXiv:2308.08155* (2023).

[46] Zhiheng Xi et al. “The rise and potential of large language model based agents: A survey”. In: *arXiv preprint arXiv:2309.07864* (2023).

[47] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *arXiv preprint arXiv:2210.03629* (2022).

[48] Andy B Yoo, Morris A Jette, and Mark Grondona. “SLURM: Simple linux utility for resource management”. In: *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, 2003, pp. 44–60.

[49] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018, pp. 3911–3921.

[50] Aohan Zeng et al. “AgentTuning: Enabling Generalized Agent Abilities for LLMs”. In: *arXiv preprint arXiv:2310.12823* (2023).

[51] Lianmin Zheng et al. “Judging LLM-as-a-judge with MT-bench and chatbot arena”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[52] Victor Zhong, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. In: *arXiv preprint arXiv:1709.00103* (2017).